



ERPWARE — FOUNDATION SPECIFICATION

A tenant-neutral ERP platform · Version 1.2 · 1 September 2026

The foundational reference for the product. It defines what ERPWare is, how organisations are isolated from one another, the language it uses, the model beneath it, the guarantees it makes, the functional ground it covers, how an organisation's way of working is expressed without touching the product, and the decisions already settled.

It contains no customer names, no customer processes, no customer configuration and no customer data.

This document defines ERPWare as a product. It contains no customer names, no customer processes, no customer configuration and no customer data. Any organisation can be onboarded as a tenant without inheriting another tenant's terminology, rules, master data or history.

Deployment-specific material — a given organisation's entities, sites, legislation bindings, terminology, approval chains, process variants and migration mappings — lives outside this document, in that tenant's own deployment profile. Nothing in a deployment profile may require a change to the product.

Contents

- Part 1 — Product definition
- Part 2 — Tenancy and isolation
- Part 3 — Naming and vocabulary
- Part 4 — Platform kernel
- Part 5 — Canonical domain model
- Part 6 — Correctness and consistency
- Part 7 — Functional scope
- Part 8 — Extension model
- Part 9 — Operations, assurance and lifecycle
- Part 10 — What the product refuses
- Part 11 — Build sequence
- Part 12 — Starter content packs
- Part 14 — Device operations
- Part 15 — Output and communications
- Part 16 — Release and environments
- Part 17 — Support and incident management
- Part 18 — Commercial and entitlement
- Part 19 — Reporting and analytics services
- Part 20 — Data migration and import
- Part 21 — Accessibility and inclusive design
- Part 22 — Guidance and adoption
- Part 23 — Decisions register

PART 1 — PRODUCT DEFINITION

1.1 What ERPWare is

An ERP platform for multi-entity, multi-site organisations that hold, move, transform and sell physical goods under regulatory obligation. It covers master data, inventory, procurement, planning, production, sales and fulfilment, finance, quality and traceability, on an event-sourced core designed so that behaviour is configured rather than coded.

1.2 Target organisational profile

- Multiple companies, multiple currencies, multiple jurisdictions
- Multiple physical sites, with warehousing and some level of production, assembly or repackaging
- Batch or lot control with expiry, and traceability obligations that must be answerable on demand
- Multiple demand channels, including automated order intake from upstream systems
- Finance requiring dimensional analysis, statutory reporting per jurisdiction, and consolidation

1.3 Design objectives

- O1 Configuration over code: an organisation's way of working is expressed as data, not development. Two tenants with different processes run the same binary
- O2 Complete tenant isolation: no tenant can observe, infer or inherit another tenant's data, configuration or terminology
- O3 Auditability by construction: every state is derived from an immutable event history; every change is attributable and reversible
- O4 Regulatory defensibility: traceability, approval control, and evidence generation are properties of the platform, not per-customer builds
- O5 Coexistence and migration: the platform can run alongside an incumbent ERP, reconcile against it, and assume domains from it progressively
- O6 Change safety: any configuration change can be previewed, promoted, and reverted without downtime

1.4 Product boundary

In scope: master data, inventory and warehouse, procurement, supply chain planning, production, sales and order management, finance, quality and traceability, logistics, reporting, workflow, administration.

Out of scope: HR and payroll, CRM and marketing automation, clinical or prescribing systems, storefront commerce, finite-capacity scheduling, treasury and hedging instruments.

PART 2 — TENANCY AND ISOLATION

This part exists because the product must be usable by any organisation with no risk of another organisation's material being present. It is foundational: tenancy is not a feature added later, it is the outermost scope of every object in the system.

2.1 Tenancy model

- **tenant** is the root scope. Every business object, configuration object, event, document, audit record, file and job carries a tenant identifier — there is no unscoped operational table
- Below tenant sits the organisational hierarchy: tenant → entity (legal) → site (physical or logical) → location. Scoping rules cascade down this hierarchy and never across tenants
- Users belong to exactly one tenant. Cross-tenant identity does not exist; a person needing access to two tenants holds two accounts
- Isolation model: logical isolation by default (shared schema, tenant-scoped rows, row-level security enforced in the database) with the option of physical isolation (dedicated schema or dedicated database) per tenant where a customer's regulatory posture requires it. The application code is identical in both cases

2.2 Enforcement

- Row-level security policies on every table, keyed on the session's tenant context, applied at the database layer so that an application bug cannot leak data
- Tenant context is derived from the authenticated session only — never from a request parameter, header or client-supplied value
- Every query path, background job, export, report and integration call runs inside a tenant context; jobs without one cannot start
- Automated isolation testing: the test suite includes adversarial cases attempting cross-tenant reads and writes through every entry point, and these run on every build
- Storage isolation: files, documents, print outputs and exports are stored under tenant-scoped paths with tenant-scoped access tokens
- Encryption: per-tenant data encryption keys, so a key compromise or a key revocation is bounded to one tenant

2.3 Product content versus tenant content

The line that makes the product genuinely reusable:

- Product content ships with the release and is identical for every tenant: the data model, rule engines, state machine definitions, event types, standard reports, legislation packs, language packs, and neutral starter templates
- Tenant content is created within a tenant and never leaves it: entities, sites, master data, transactions, events, terminology overrides, approval chains, thresholds, mappings, custom reports, and process variants
- Starter templates are neutral by construction: a starter chart of accounts, a starter set of reason codes, a starter approval chain — generic, illustrative, derived from standards or common practice, never from a customer's configuration
- No customer-derived artefact may be promoted into product content. If a tenant's configuration suggests a good default, the pattern may be generalised, but the artefact itself is rebuilt from neutral inputs and reviewed for identifying content

2.4 Onboarding a tenant

- Provision: create tenant, select legislation and language packs, apply neutral starter templates, create the first administrator
- Configure: entities, sites, locations, users and roles, then the configuration domains the organisation needs (approval chains, policies, terminology, document templates)
- Migrate: load master data and opening balances through the controlled import pipeline
- Validate: run the conformance suite for the selected legislation packs, plus the tenant's own acceptance fixtures

- No engineering involvement is required for any of the above. If onboarding needs code, that is a product gap, logged as such

2.5 Tenant lifecycle and data rights

- Export: complete tenant data export in open formats, on demand, including events and audit history
- Portability: exports are structured so a tenant can leave without a bespoke extraction project
- Deletion: full tenant deletion with cryptographic key destruction and verifiable removal, subject to retention obligations the tenant specifies
- Residency: tenant data can be pinned to a region; the product does not assume a single deployment location
- Backup and restore operate at tenant granularity, so one tenant can be restored without touching another

PART 3 — NAMING AND VOCABULARY

Three vocabularies operate in this product and confusing them is the most common source of ambiguity when reading a specification back. This part fixes which is which.

3.1 The three vocabularies

- **Model vocabulary** — how data is structured: `tenant`, `party`, `item`, `document`, `principal`, `aggregate`. Correct in a schema, never shown to a user. Model names do not change when product language does, because renaming tables buys nothing and costs migrations
- **Product vocabulary** — what the base locale shows every organisation. This is standard UK ERP language, because that is what a UK buyer, operator or auditor already knows
- **Tenant vocabulary** — what a given organisation calls things, applied as overrides through the terminology layer in §4.10

Because every user-facing string resolves through a resource key, the product vocabulary is data. It can be changed without a release. That is precisely why the base locale must be right at the outset: it is what every new organisation inherits before it overrides anything.

3.2 Product vocabulary — the decisions

Where the model name and the product name differ, the product name wins on screen:

- `tenant` → **organisation**
- `entity` → **company**. Entity was doing two jobs — a legal company and any data object — and that dual use was the largest single source of ambiguity
- `principal` → **user**, and a non-human user is a **service account**
- `party` → **business partner**, narrowed to **supplier** or **customer** wherever only one role is meant
- `item` → **product**, and **stock item** where the stockable subset is specifically meant
- `container` → **handling unit**, the industry-standard term for the recursive pallet-carton-tote structure, and unambiguous where container suggests shipping
- `posting class` → **accounting code**, the term a UK finance user already has for exactly this concept
- `account` → **nominal account**; the chart of accounts is also the **nominal ledger**. General ledger is retained as a recognised alias
- `fiscal period` → **accounting period**; `fiscal calendar` → **financial calendar**; fiscal year → **financial year**. Fiscal is US usage
- `dimension` → **analysis code** on screen, **analytical dimension** in formal writing
- Accounts receivable and payable → **sales ledger** and **purchase ledger**, with AR and AP retained as aliases
- Staging for despatch → **marshalling area**, the established UK warehouse term
- Receiving → **goods-in**; despatch staging → **goods-out**

3.3 Terms kept deliberately

These are already correct UK usage; changing them would move away from standard:

- **Stock**, not inventory, in operational language. Inventory survives as a module name for the whole domain, but the ledger is a stock ledger and the count is a stock count
- **Works order**, not work order or production order
- **Despatch**, not dispatch
- **Batch**, not lot. UK and EU regulated practice says batch; lot is retained as an alias
- **Supplier**, not vendor
- **Delivery note**, **picking list**, **goods received note**, **purchase requisition**
- **GRNI** — goods received not invoiced; the US GR/IR is not used
- **Cycle count** for the perpetual programme, **stocktake** for the wall-to-wall annual. These are distinct operations and UK practice distinguishes them
- **Global allocation** and **detailed allocation** — established vocabulary in mid-market ERP, and understood without translation by the people most likely to configure it
- **Qualified Person** and **Responsible Person** — correct regulatory titles, never softened
- **Available to promise**, **pegging**, **firming**, **time fence**, **safety stock**, **reorder point** — standard planning vocabulary, internationally consistent

3.4 Model vocabulary — never on a screen

These are correct in this document and in the schema, and carry no meaning to an operator:

`aggregate` · `event envelope` · `outbox` · `projection` · `upcaster` · `document spine` · `write gateway` · `capability` (say **feature**) · `change set` (say **change** or **release**) · `posting rule` (say **accounting rule**) · `determination matrix` (say **account determination**) · `party role terms` (say **trading terms**)

3.5 Terms defined once, because they are genuinely ambiguous

Each of these means different things in different systems. The base glossary states the product's meaning rather than assuming it:

- **Site** — a physical or logical operating location under a company. Not a warehouse only, and not a legal establishment
- **Location** — a storage position within a site: zone, aisle, rack, bin. Not a geographic place; where that is meant, say **address**
- **Allocation** — reserving stock against demand. The accounting sense of apportioning cost is called **cost apportionment**, never allocation
- **Release** — three unrelated senses, always qualified: **order release** to the warehouse, **batch release** from quarantine, **promotion** of a change to live
- **Validation** — two senses, always distinguished: **confirm delivery** for the operational act, and validation reserved for computerised system validation in the regulatory sense
- **Class** — always qualified: accounting code class, product class, count class, hazard class. Never used alone

3.6 Locale strategy

- The base locale is **en-GB**, carrying the vocabulary above
- Alternative terms are seeded as glossary entries rather than discarded, so an organisation preferring item, vendor or GL account overrides a handful of keys and stops thinking about it
- An **en-US** variant ships with the product, reversing only what genuinely differs: inventory for stock, work order for works order, dispatch for despatch, lot for batch, vendor for supplier, accounts receivable for sales ledger. That variant also serves as the working proof that the fallback chain in §4.10 behaves
- No user-facing literal exists anywhere in code. A term that cannot be overridden is a defect

PART 4 — PLATFORM KERNEL

Shared services every functional module consumes. Built once, tenant-scoped throughout.

4.1 Identity and access

- Authentication via the tenant's chosen identity provider; service accounts for integrations and jobs
- Role-based access with permissions at module and action granularity, scoped by entity, site and, where needed, data class
- Segregation-of-duties rule definitions, with conflict detection and periodic access review packs
- All access decisions logged and reportable

4.2 Audit

- One append-only audit stream per tenant: actor, action, object type and identifier, before and after state, timestamp, correlation identifier
- Sensitive reads are auditable, not only writes
- No role, including administrator, holds update or delete rights on audit data

4.3 Event bus and event store

- Append-only event store per tenant, with an outbox per module for reliable publication
- Event envelope: identifier, versioned type, occurred-at, recorded-at, tenant, entity, site, aggregate type and identifier, actor, correlation and causation identifiers, schema-validated payload, source
- Consumers hold their own cursors; replay from any point is a supported operation
- Events are past-tense facts, never commands, and are immutable once recorded

4.4 Configuration engine

- Configuration objects are typed, versioned, effective-dated, tenant-scoped, with full change history
- Domains: organisational structure, policies, rules, thresholds, mappings, templates, approval chains, state machines, terminology, report definitions, integration definitions
- Configuration is data, so it is queryable, diffable, promotable and revertible

4.5 Rule engine

- Declarative rules: condition, evaluation order, outcome, and effect on documents, postings or workflow

- Rules are versioned and effective-dated; the version in force at a transaction's date is the version applied, permanently
- Every rule evaluation is traceable: given an outcome, the system can state which rule, which version, which inputs
- No business rule is expressed in application code where a tenant might reasonably need it to differ

4.6 State machine engine

- Document and object lifecycles are defined as configuration: states, permitted transitions, guards, required roles, side effects
- Transition validation prevents unreachable states, orphaned states and undeclared transitions — the class of defect that produces blind spots in hand-coded status logic
- State machines are versioned; in-flight documents complete under the definition they started with

4.7 Workflow and approvals

- One approval engine consumed by every module: chains defined by scope, value band, category and object type; sequential, parallel and conditional steps; delegation, substitution and escalation on ageing
- Re-approval rules on material change, with configurable tolerance
- Complete approval audit, including the version of the object approved

4.8 Scheduler and jobs

- Scheduled and triggered execution with timeout, retry policy, overlap protection and concurrency limits per tenant
- Planned-outage calendar suppresses jobs and their alerts during known windows
- Every run logged with outcome and summary counts; a job that stops running raises an alert

4.9 Notifications

- Channel abstraction with routing rules as configuration, digesting, escalation and quiet hours
- Templates are resource-based and localised

4.10 Localisation

- Legislation packs: versioned, effective-dated bundles of parameters, declarative rules and statutory output definitions, shipped with the product and bound per entity. Adding a jurisdiction is configuration, not installation, and never a code branch
- Language and terminology: no user-facing literal anywhere; every string resolves through a resource key. Locales with fallback chains so a variant supplies only genuine differences. Separate axes for user language, document language and reporting language
- Both pack types are product content, neutral and shared; a tenant's overrides are tenant content

4.11 Change promotion and rollback

- Changes accumulate as named change sets with diffs against the target environment
- Promotion is previewed, gated by approval, validated by tests, scopeable and scheduled, and requires no downtime
- Rollback restores a versioned snapshot in one action, with blast-radius preview; a kill switch disables an individual rule, job or command class immediately
- Environments cannot silently diverge: direct edits to live configuration are not permitted, and drift is detected continuously

4.12 Configuration intelligence

- Assists authoring, explanation, impact analysis, test generation, onboarding interviews and migration mapping
- Proposes, never applies: every suggestion is a diff requiring named human approval, validated in a test environment first
- Never in the transaction path: postings, allocations, tax determination and stock movements are executed by deterministic rules only
- Operates strictly within tenant scope; no cross-tenant learning, no cross-tenant examples, no model artefacts trained on one tenant's data being used to serve another

PART 5 — CANONICAL DOMAIN MODEL

The entities an ERP requires, their relationships, and the invariants that must hold. This is the part that must be right before anything is built on it.

5.1 Organisation

- **tenant** — the isolation root
- **entity** — company; base currency, legislation bindings, financial calendar, tax registrations, reporting language
- **site** — physical or logical operating location under an entity; type, timezone, calendar, licences

- **location** — structure within a site: hierarchical (zone, aisle, rack, bin), typed (receiving, bulk, pick, quarantine, staging, despatch, damages, production), with capacity, storage conditions and count classification
- **calendar** — working days, shifts, holidays; drives all lead-time and date arithmetic
- Invariant: every operational object resolves to exactly one entity and, where physical, exactly one site

5.2 Parties

- **party** — a single identity that may hold multiple roles (supplier, customer, carrier, internal, regulator contact); roles carry their own attribute sets rather than duplicating the identity
- **party_address** and **party_contact** — typed, effective-dated
- **party_role_terms** — payment terms, currency, tax rule, credit limit and status, price list membership, incoterms, approval status
- Invariant: a party's identity is stable across roles and entities; commercial terms are per role, per entity

5.3 Products

- **item** — identity, descriptions (localised), classification, lifecycle status, control flags (batch, expiry, serial, quarantine-on-receipt, FEFO), regulatory attributes, barcodes
- **uom** and **uom_conversion** — stock, purchase, sales and packing units with conversions; catch-weight support
- **item_site** — per-site planning and control parameters
- **item_price** — price lists and cost records with validity ranges
- **bom** and **bom_line** — versioned, effective-dated, multi-level, with yield and scrap factors, alternates and substitutes
- **routing** and **routing_operation** — resources, times and rates
- Invariant: an item's control flags may not be changed while stock exists in a state the new flags cannot represent

5.4 Stock

- **batch** — lot identity with attributes (manufacture, expiry, retest, supplier lot, origin, certificates) and a status lifecycle; attributes are amendable through evented amendment with approval, without generating stock movements
- **serial** — where serialisation applies, with genealogy
- **container** — recursive handling unit: type, identity, parent container, location, contents. Nesting depth is unbounded; identity depth is a configured policy per product class and process step. Moving a container moves everything beneath it as one event
- **stock_ownership** — owner party and custody party carried on every position and movement, defaulting to the operating entity. Ownership drives valuation and balance sheet inclusion; custody drives operational visibility and counting. The two are independent, so stock owned but not held (third-party logistics) and stock held but not owned (consignment inbound, contract manufacturing) are both first-class rather than exceptions
- **stock_condition** — free, bonded or duty-suspended, consignment, quarantined, blocked; a fiscal and operational qualifier independent of location
- **cost_layer** — where a layered costing method applies: acquisition cost, quantity, consumption ordering and remaining balance, with revaluation events against layers
- **stock_movement** — the append-only ledger: typed movement, item, batch, serial, container, owner, custody, condition, from and to location, quantity, unit, cost context, document reference, reason code, actor, occurred-at, recorded-at
- Derived positions — on hand, available, allocated, in transit, quarantined, blocked, in production — are computed from the ledger and never stored as an editable balance
- **allocation** — two-stage: quantity-level reservation against a scope, and stock-line-level commitment to specific batch, location and container, evaluated under configured policy
- Invariants: on-hand quantity equals the sum of movements per owner and custody, always; inventory value derived from cost layers reconciles to the control account at any point in time; corrections are reversing movements, never edits; negative on-hand is impossible except through explicitly authorised movement types; a batch's genealogy is continuous and unbroken by any record correction

5.5 Production

- **work_centre** — a resource or group that performs operations: capacity basis, calendar, cost rates
- **works_order** — a production, assembly, kitting, rework or repackaging order: type, item, quantity, dates, source demand, state
- **works_order_component** — planned and actual consumption per component, with the batches issued
- **works_order_operation** — planned and booked execution per routing step: quantity completed, scrapped with reason, labour and machine time, operator
- **batch_record** — the assembled execution history for a produced batch: materials and batches consumed, operations, operators, timestamps, equipment, deviations, and the qualified release
- **production_variance** — material price and usage, yield, labour rate and efficiency, overhead absorption, per order and per period
- Invariants: issued quantity less consumed less scrapped reconciles to output within configured yield tolerance; a finished batch's genealogy resolves to every component batch consumed; a works order cannot close with unreconciled material; a batch record is complete before qualified release is permitted

5.6 Documents

- **document** — a generic spine: type, tenant, entity, site, party, state, dates, currency, references, external references

- `document_line` — item, quantity, unit, price, dimensions, batch and location commitments, line state
- `document_relation` — typed lineage links between documents (order to delivery, delivery to invoice, requisition to order, order to works order, return to credit)
- Document types are configuration on the spine: requisition, purchase order, receipt, return to supplier, quotation, sales order, delivery, invoice reference, credit reference, transfer order, works order, adjustment, count
- Each type binds to a state machine definition, an approval configuration and a numbering rule
- Invariants: lineage is navigable in both directions; a document's state history is complete; no document may be deleted, only cancelled or reversed with reason

5.7 Planning

- `forecast` and `forecast_version` — by item, site, channel and period bucket, with method, parameters and accuracy history
- `planning_policy` — reorder method, safety stock basis, service level, lot sizing, time fences, sourcing rules
- `planned_order` — proposed purchase, transfer or production, with pegging to the demand that created it
- `planning_exception` — typed, aged, actionable
- Invariant: every planned order traces to its demand source; firming converts a plan to a document without losing the link

5.8 Finance

- `account` — chart of accounts per entity with group mapping; type, control account designation, status
- `ledger` — parallel ledgers (statutory, group, tax, management) with their own calendars and rules
- `fiscal_period` — per ledger and entity, with status
- `dimension` and `dimension_value` — analytical axes; rules define which are mandatory per account, how they are derived from source events, and which combinations are permitted
- `journal` and `journal_line` — source-coded, with the posting rule and rule version recorded on every line
- `posting_rule` — declarative mapping from an operational event to accounts and dimensions, per entity and legislation
- `subledger_item` — payable, receivable, inventory and asset detail reconciling to control accounts
- `tax_determination` — rule-driven, with the determination recorded on the transaction
- Invariants: every journal balances by ledger and currency; every posting traces to an operational event and the rule version that produced it; a closed period cannot be posted to without an explicit reopening event; subledger totals equal their control accounts at all times

5.9 Quality and compliance

- `inspection` and `inspection_result` — plans, sampling, results, disposition
- `quality_event` — deviation, non-conformance, complaint, excursion, with investigation and corrective action tracking
- `recall` — scope by batch or criteria, impacted despatch set, action log, reconciliation of recovered and unaccounted quantity
- `release_record` — authorised release of a batch by a qualified role, with signature and basis
- Invariant: given any batch, the complete downstream despatch set and upstream component set are retrievable as a query, at any time

5.10 Integration

- `external_system` — a registered counterpart system with its adapter configuration
- `external_ref` — a mapping between an ERPWare object and its identifier in an external system, with sync state
- `integration_message` — inbound and outbound message log with status, retries and payload reference
- `command` — an outbound intent through the write gateway: target, payload, idempotency key, approval state, dry-run flag, lifecycle events
- Invariant: all outbound writes pass through the gateway; no module holds credentials or calls an external system directly

5.11 Cross-cutting model rules

- Identifiers: internal surrogate keys are opaque; human-facing numbers come from configured sequences; external identifiers live only in `external_ref`
- Quantities carry their unit; money carries its currency and is stored in minor units; no floating point in either
- Time: events record both when something happened and when it was recorded; storage is absolute, display is local
- Every business object carries created and modified attribution; nothing meaningful is deleted, only superseded, cancelled or reversed

PART 6 — CORRECTNESS AND CONSISTENCY

An event-sourced core is only as trustworthy as the guarantees around it. This part states them explicitly, because each is a place where a system can be internally consistent and still produce a wrong answer.

6.1 Concurrency control

Append-only storage does not by itself prevent two writers acting on stale state. Two allocations that each read one unit available will each succeed, the ledger will

remain internally consistent, and the business answer will be wrong.

- Every aggregate stream carries a version. Appending supplies the expected version and fails on mismatch
- Concurrency boundaries are declared, not incidental. The minimum set: stock at item-batch-location, a document, a batch's status, an allocation, a numbering sequence, a period's open state
- Conflict is retried under a bounded policy; operations that cannot safely retry are serialised at the boundary rather than merged
- A derived position may not be read outside the transaction that acts on it. Read-then-act across transaction boundaries is a defect class, not a pattern

6.2 Event schema evolution

Replay from any point is promised. That promise fails silently at the first payload change unless evolution is designed.

- A schema registry holds every event type and version, with additive-only compatibility within a major version
- Old events are read into current shape by an upcaster chain at consumption time; stored events are never rewritten
- Deprecation removes an event type from emission, never from readability
- Every build replays the oldest retained event through current consumers as a test

6.3 Time, backdating and restatement

Recording both when something happened and when it was recorded is the hard part, already in the model. What must also be defined is what the system does with the distinction.

- Whether an event may be recorded with an occurrence date inside a closed period is configuration per document class: refuse, accept with restatement, or route to the next open period
- Maximum backdating window is configured, not assumed
- Two query modes are supported and distinguished: the position as it actually was, and the position as it was believed to be at a past moment. Comparatives, audit and restatement need the second
- A correction to a prior period leaves both the original and corrected views retrievable. History is never rewritten

6.4 Numbering under failure

Uniqueness is easy; gaplessness is hostile to transactions, retries and rollbacks, and some jurisdictions require it.

- Each sequence declares whether it is merely unique or statutorily gapless
- Gapless numbers are allocated on commit, never on attempt
- A cancelled document retains its number against a cancellation record; numbers are never reused
- Per-establishment and per-period series are supported where a legislation pack requires them

6.5 Idempotency on both edges

- Outbound commands carry idempotency keys through the gateway
- Inbound API writes and integration messages carry them too, with a declared deduplication window, so a retried intake cannot create a second order

6.6 Projections and read performance

Deriving state from events is correct as a model and unusable as a query strategy at volume.

- Hot reads are served by maintained projections: on-hand and available, allocation position, document state, ageing balances
- Every projection has a rebuild-from-events command and a standing reconciliation check comparing it to the ledger. The invariant is unchanged; the projection is how it is served and the check is how it is proven
- The event store is partitioned by tenant and period, with a snapshot policy per aggregate and an archival tier that preserves both replay and retention obligations

6.7 Consistency guarantees, stated

Eventual consistency is only acceptable when bounded and disclosed.

- Read-your-own-writes is guaranteed for the acting user
- Anything that gates a commit — allocation, approval, posting, period state — is strongly consistent by construction
- Maximum projection lag is bounded and monitored; a read beyond the bound is served with an explicit staleness indication rather than silently

6.8 Failure semantics at the boundary

- A command whose outcome is unknown enters an explicit ambiguous state with a defined reconciliation procedure. It is never blindly retried
- Poison messages are quarantined after a declared attempt count rather than cycling
- A failed compensation escalates to a human queue; it does not retry indefinitely

6.9 The invariants, as executable assertions

These are product artefacts, run against the whole database after every suite, not prose:

- On-hand equals the sum of movements, per item, batch, location and owner
 - On-hand is never negative except through explicitly authorised movement types
 - Corrections exist only as reversing movements; no movement is edited or deleted
 - Every journal balances by ledger and currency
 - Subledger totals equal their control accounts
 - Every posting traces to an operational event and the rule version that produced it
 - A batch's genealogy is continuous across amendment, split and merge
 - Document lineage is navigable in both directions and complete
 - No closed period holds a posting recorded after closure without an explicit reopening event
 - No user holds identity in more than one tenant
 - No query path returns a row belonging to another tenant
-

PART 7 — FUNCTIONAL SCOPE

Each module lists purpose, functional scope by sub-function, the data it owns, its configuration surface, the events it publishes, and the an incumbent system functions it eventually displaces. Phase markers: Observe (monitor and reconcile), Assist (workbench and controlled actions), Own (ERPWare is system of record). Functional scope is defined in full even where delivery is years out.

7.1 MDM — Master Data Management

Purpose: one governed source for the reference data every other module depends on.

7.1.1 Product master

- Item identity: code, descriptions (short/long, multilingual), product category and family, status (active/obsolete/pending), lifecycle dates
- Classifications: product type (drug, device, packaging, consumable, non-stock, service), therapeutic/commercial groupings, ABC/XYZ classification, hazard and controlled-drug flags
- Control flags: batch-managed, expiry-managed (with shelf-life rules and UBD offset), serialised, quarantine-on-receipt, FEFO-enforced, weighed/catch-weight
- Units of measure: stock UoM, purchase UoM, sales UoM, packing UoM with conversion factors; pack size, case size, pallet configuration (ti/hi)
- Physical attributes: weight, dimensions, storage conditions (cold chain, ambient, controlled), stackability
- Commercial attributes: standard cost, last cost, average cost reference, price list membership, tax code by entity
- Regulatory attributes: licence class, product authorisation reference, commodity code, country of origin, GTIN/EAN/barcodes
- Item-site parameters: per site — reorder policy, min/max/safety stock, lead time, count class, default locations, planning flags, valuation site

7.1.2 Business partner master

- Suppliers: identity, addresses, contacts, currency, payment terms, tax rule, approval status (approved supplier list for regulated goods), lead times, MOQ/MOV, incoterms, remittance and bank details reference, performance history link
- Customers: identity, bill-to/ship-to/sold-to structure, credit limit and credit status, payment terms, price list, tax rule, delivery instructions, channel (B2B wholesale, prepaid direct via the upstream order system, intercompany)
- Carriers: service levels, cut-off times, tracking integration reference, cost matrices
- Internal parties: intercompany trading relationships, transfer pricing rules

7.1.3 Structural master data

- Entity registry: company, base currency, legislation, financial calendar, tax registration
- Site registry: type (warehouse, pharmacy, production, virtual), timezone, address, operating hours, licence references
- Location structure: zone → aisle → rack → bin, with location types (pick, bulk, quarantine, quarantine-cold, damages, staging, goods-in, goods-out), capacity and constraints, count classes
- Calendar: working days, shifts, public holidays per entity/site — drives lead-time and planning maths

7.1.4 Data quality and governance

- Completeness and validity scoring by domain, per field weighting, owner assignment, dashboards and trend
- Duplicate detection and merge workflow with audit
- Change request workflow: request → validate → approve → apply → verify, with field-level approval rules (e.g. cost and tax fields require Finance approval)
- Controlled import pipelines: template definition, file validation, preview with error report, staged load, rollback — replaces raw an incumbent system template imports

- Mass maintenance: rule-based bulk updates with preview and reversal

Owns: item, item_site, party, entity, site, location, calendar, data quality rules
 Config: field requirements by item type, scoring weights, approval chains, import templates, validation rules
 Events: item.created/updated, party.approved, location.created, masterdata.exception.raised

7.2 Inventory & Warehouse

Purpose: the stock ledger and the physical operations that move stock, batch- and expiry-native throughout.

7.2.1 Stock ledger

- Append-only movement records: receipt, putaway, internal transfer, pick, issue, despatch, return, adjustment, count variance, scrap/destruction, production issue, production receipt, quarantine release/block, revaluation
- Every movement carries: item, batch, from/to location, quantity, UoM, cost context, document reference, reason code, actor, occurred_at and recorded_at
- Derived positions: on-hand, allocated, available-to-promise, in-transit, quarantined, blocked, in-production — by entity, site, location, item, batch
- Corrections as reversing entries with mandatory reason codes; no edits, no deletes (the structural answer to phantom stock from upstream deletion)
- Valuation: costing method per item/entity (standard, average, FIFO), cost layers, revaluation events, stock value by dimension reconcilable to the ledger at any point in time

7.2.2 Lot, batch and expiry

- Batch identity: system-generated or supplier lot, manufacture date, expiry date, UBD (use-by-date) derivation rules, retest date, country of origin, supplier lot cross-reference
- Batch status lifecycle: created → quarantine → released (qualified sign-off captured) → blocked/recalled → consumed/expired
- Shelf-life rules: minimum remaining shelf life on receipt, on despatch (customer-specific), on transfer; violations blocked or exception-queued
- Expiry management: horizon views (30/60/90 day), expiring-stock exception queue, write-off proposal workflow, expiry-driven markdown or return-to-supplier triggers
- Genealogy: batch → all movements → all documents → all customer despatches, both directions, as a query (recall readiness measured continuously against configured regulatory clocks)
- Serialisation (where applicable): serial capture at receipt/despatch, serial genealogy

7.2.3 FEFO and allocation

- Allocation engine: policy per item/site/channel — FEFO (default for expiry-managed), FIFO, specific batch, nearest-location, single-batch-per-order constraint
- Shelf-life-aware allocation: exclude batches failing customer minimum-remaining-life
- Reservation model: soft (planning) vs hard (picked) allocation; reservation ageing and auto-release
- Substitution rules: alternative items/pack sizes with approval
- Shortage handling: shortage queue with cause classification (no stock, quarantined, allocated elsewhere, expiry-blocked), auto-retry on replenishment, escalation thresholds
- Allocation override with reason and approval, fully audited

7.2.4 Warehouse operations

- Goods-in: booking-in schedule, ASN handling, unload, count-in, quarantine placement, GRN creation, discrepancy capture (short, over, damaged, wrong item, temperature excursion)
- Putaway: rule-driven location suggestion (zone, capacity, storage conditions, velocity), directed or confirmed, pallet/container ID container identity handling for mixed and single-lot pallets
- Replenishment: bulk-to-pick location top-up rules, min/max at pick face, wave-driven or continuous
- Picking: wave/batch/zone/piece picking strategies, pick list generation, handheld-directed picking, short-pick handling, pick confirmation with batch/serial capture
- Packing: carton selection, pack verification (scan-to-verify against order), consolidation, weight capture
- Despatch: load building, manifest, courier hand-off with scan (no dispatch without a courier scan), despatch confirmation events
- Returns inbound: return receipt, inspection, disposition (restock, quarantine, destroy), credit trigger

7.2.5 Stock counting

- Count programmes: perpetual/cycle counting by ABC class and count frequency, annual wall-to-wall, spot counts, location sweeps, count-on-zero triggers
- Count scheduling: generated schedules by class/zone/value, workload balancing, blackout windows
- Count execution: count sheets or handheld-directed, blind vs informed counting, multi-count (recount thresholds), batch- and pallet-level counting methods (including mixed-lot pallet method)
- Variance management: variance calculation by quantity and value, tolerance rules, investigation workflow, approval thresholds by value, root-cause coding
- Posting: approved variances become adjustment movements with reason codes; rejected counts recount
- Reporting: stock accuracy % by site/class/location, variance trend, count coverage compliance, value at risk

7.2.6 Inventory control and analytics

- Stock health: ageing, slow-moving and obsolete identification, dead stock value, cover in days
- Stock accuracy KPI framework and inventory record accuracy reporting
- Write-off and scrap workflow with authorisation levels and reason analysis
- Stock valuation reporting by entity/site/category, reconciliation to the GL stock accounts
- Constraint checks: negative stock prevention (movement-type-authorised only), overallocation prevention, location capacity

7.2.7 Counting without freezing operations

The problem: a count session locks stock, blocking movements and disrupting allocation and sales order processing while it runs. Counting therefore gets deferred, done out of hours, or avoided — and stock accuracy suffers for the sake of not interrupting fulfilment.

Root cause: an incumbent system protects count integrity by preventing concurrent change. That is only necessary if the count is a snapshot compared to a live balance at posting time. An event-sourced ledger removes the need.

MOVEMENT-TOLERANT COUNTING

- A count captures counted quantity with a timestamp (`counted_at`), not a frozen balance
- At posting, the system reconstructs expected quantity as at `counted_at` from the movement ledger, calculates the variance against that point in time, then replays any movements recorded between count and posting on top
- Result: operations never stop. A pick that happens mid-count is not a variance, it is a movement — and the maths knows the difference
- Requires no lock at all for the common case

GRANULAR LOCKING WHERE A LOCK IS GENUINELY NEEDED

- Lock scope is never site-wide or item-wide. The narrowest applicable scope only: a specific location, container, or batch-at-location
- Soft lock by default: movements against a soft-locked scope are permitted but flagged for count adjudication (the counter sees “3 units picked during your count”)
- Hard lock available for high-value or regulatory counts, but time-boxed with automatic expiry and an audit record of what it blocked
- Priority override: fulfilment can always break a soft lock; the count adjusts rather than the order failing

COUNT SCOPE INTELLIGENCE

- Automatic exclusion of stock already allocated, picked, or staged for despatch — counting what is on its way out creates false variances and blocks orders for no gain
- Count scheduling aware of the order book: locations feeding today’s release wave are scheduled outside the wave window
- Count-on-zero and opportunistic counting: when a pick empties a location, that location is auto-counted with no separate session

VARIANCE HANDLING THAT DOES NOT DISRUPT

- Variances post as adjustment movements with reason codes, after tolerance and approval rules — never as a stock freeze pending investigation
- Investigation runs as a workflow alongside live operations, on the variance record, not on the stock
- Recount triggers apply to the location only

Configuration: lock mode by count type, tolerance bands, exclusion rules, scheduling windows, approval thresholds, recount triggers
Events: count.session.opened, count.recorded, count.variance.calculated, count.variance.approved, stock.movement.recorded (adjustment)

7.2.8 Batch and expiry amendment without out-and-in

The problem: correcting an expiry date, UBD, or lot attribute requires issuing the stock out and receiving it back in as fresh stock. This is laborious, it fabricates movement history that never physically happened, it disturbs cost and valuation, and it destroys the genealogy that a recall depends on.

Root cause: expiry and lot data being treated as a property of the receipt rather than of the batch.

BATCH ATTRIBUTES ARE AMENDABLE, STOCK IS NOT MOVED

- Batch is a first-class entity with its own attribute set (expiry, UBD, retest date, supplier lot, country of origin, status, certificates)
- Amending an attribute writes a `batch.attribute.amended` event — before value, after value, reason code, actor, approver, timestamp, supporting document reference
- Zero stock movements are generated. Quantity, location, cost and valuation are untouched. Genealogy remains intact and continuous

CONTROLS PROPORTIONATE TO THE FIELD

- Field-level rules: which attributes are amendable, by which role, with what approval. Expiry extension on a regulated product requires the qualified authority or RP approval with a documented basis; correcting a mistyped supplier lot needs a supervisor
- Amendments are never silent: the batch record shows full amendment history, and any document, report or recall pack citing that batch shows the value in force at the time alongside the current value
- Directional guards: shortening expiry may be permitted freely; extending it is restricted and always approval-gated
- Downstream recalculation is automatic — FIFO ordering, expiry-horizon views, shelf-life-on-despatch checks and expiring-stock queues all reflect the amendment immediately

RELATED BATCH OPERATIONS, ALSO WITHOUT OUT-AND-IN

- Split: divide a batch into sub-batches (part quarantined, part released) as a batch operation with lineage to the parent
- Merge: combine batches of identical item and attributes where operationally justified, with lineage retained to both parents
- Re-status: quarantine, release, block, unblock — status events with approval, no movement
- Re-date on retest: new expiry from a retest result, recorded against the certificate
- Genuine physical rework (relabelling, repacking) remains a works order in Manufacturing — the distinction is whether the goods changed, not just the record

Configuration: amendable fields by product category and legislation, approval matrix by field and direction, mandatory reason codes, evidence requirements

Events: batch.attribute.amended, batch.split, batch.merged, batch.status.changed

7.2.9 Storage and container identity, configurable

The problem: An organisation changes how identity is captured — from individual identifiers per box to a master identifier covering the pallet, with mixed-lot handling and a revised count method. In most systems this is a development project; the requirement is that a change of this kind is configuration.

Root cause: container identity depth being embedded in code and screens rather than modelled as a hierarchy with configurable depth.

RECURSIVE CONTAINER MODEL

- **container** is a first-class recursive entity: type (master pallet, pallet, cage, tote, carton, box), unique identity, parent container, location, status, and contents
- Contents are stock lines (item, batch, quantity) held at whatever level they are recorded — a container may hold stock directly, hold child containers, or both
- Nesting depth is unlimited by design: box inside pallet inside master pallet works without schema change, as does a flat pallet with loose contents
- Moving a container moves everything beneath it as a single event, with movements derived for every affected stock line

IDENTITY POLICY AS CONFIGURATION

- Per product category, site, and process step, configure the level at which identity is captured: none, unit, carton, pallet, or master pallet
- Configure whether mixed lots are permitted on a container, and at which level a mixed-lot container records its batch composition
- Configure identity generation: system-generated (SSCC-compatible), scanned from supplier, or derived from the parent
- Switching from per-box identity to master pallet identity is a configuration change with an effective date — historical containers keep the policy under which they were created, so the past stays readable

COUNT METHOD FOLLOWS THE IDENTITY POLICY

- Count method configurable per identity level: count containers (verify identity and expected contents without opening), count contents (open and count units), or hybrid (verify identity, spot-check contents by sampling rule)
- Mixed-lot containers get a defined method — count by batch within container, or verify container composition against system record
- The count method changes when the identity policy changes, without new development, because both are configuration bound to the same policy object

PUTAWAY AND STORAGE RULES ALSO CONFIGURABLE

- Location suitability rules by storage condition, capacity (by units, weight, volume, or container slots), and item constraints
- Container-aware putaway: suggest locations that accept a full pallet or master pallet, versus pick-face locations that take cartons
- Replenishment rules that break a master pallet down into child containers as stock moves toward the pick face, with the hierarchy updating automatically

Configuration: container types, identity level by product category and step, mixed-lot rules, identity generation, count methods, location capacity and suitability, putaway and breakdown rules

Events: container.created, container.moved, container.nested, container.broken_down, container.contents.changed

Owens: stock_movement, batch, allocation, count programmes and results, container/pallet identity

Config: costing method, allocation policies, shelf-life rules, count classes and frequencies, tolerance thresholds, putaway/replenishment rules, reason codes

Events: stock.movement.recorded, batch.released, batch.blocked, allocation.made/released, count.scheduled/completed/variance.approved, expiry.threshold.breached

7.3 Procurement

Purpose: full purchase-to-receipt lifecycle with approval control, supplier performance, and clean AP handoff.

7.3.1 Sourcing and supplier management

- Approved supplier list with qualification status, audit dates, licence checks (critical for regulated goods)
- Supplier-item catalogue: agreed price, MOQ, MOV, lead time, pack size, incoterms, validity dates, tiered pricing
- RFQ workflow: request, supplier responses, comparison, award (later phase)
- Contract and agreement register: framework agreements, price validity, volume commitments, rebate terms
- Supplier performance: OTIF, quality (rejection/return rate), invoice accuracy, lead-time reliability, scorecard by period

7.3.2 Requisition and approval workflow

- Requisition capture: requester, cost centre, item or free-text, quantity, need-by date, justification, attachments

- Budget check against cost centre/department where budgets are configured
- Approval chains: configurable by entity, site, value band, category, cost centre — sequential, parallel, or conditional; delegation and out-of-office substitution; escalation on ageing
- Approval actions: approve, reject with reason, request information, partial approval by line
- Full approval audit: who, when, on what version of the requisition; re-approval triggers on material change (value tolerance configurable)

7.3.3 Purchase orders

- PO creation: from approved requisition, from replenishment suggestion, from MRP, or manual; multi-line, multi-delivery-date, multi-site delivery
- PO types: standard, blanket/call-off, consignment, intercompany, drop-ship, service/non-stock
- Pricing: from supplier-item catalogue with override control, currency, discounts, charges, expected landed cost components
- PO issue: transmission (email/EDI/portal), acknowledgement capture, revision control with version history and re-approval rules
- Expediting: due-date monitoring, supplier confirmation tracking, chase workflow, late-delivery exception queue
- Amendment and cancellation with approval and audit

7.3.4 Receipt and matching

- Expected receipts view; ASN ingestion where available
- GRN: receipt against PO line, over/under tolerance rules, quality hold routing, batch/expiry capture at receipt, container ID assignment
- Return to supplier: RTS document, credit expectation, replacement tracking
- Three-way match: PO ↔ GRN ↔ supplier invoice, with tolerance configuration (price, quantity, total); match exception workbench with resolution actions and audit
- GRNI control: goods-received-not-invoiced register, ageing, valuation, reconciliation to the GL accrual account (the control that keeps accruals honest)
- Landed cost: freight, duty, insurance, handling — allocation rules (by value, weight, volume, quantity) onto item cost, with post-receipt adjustment handling

Owens: requisitions, POs, GRNs, RTS, match exceptions, supplier catalogue and performance Config: approval chains, tolerance rules, PO types, landed cost allocation rules, expediting thresholds Events: requisition.approved, po.issued/amended/cancelled, goods.received, rts.raised, match.exception.raised/resolved

7.4 Supply Chain Planning

Purpose: know what will be needed, when, and ensure it is there — the layer many mid-market systems serve weakly.

7.4.1 Demand planning and forecasting

- Demand history: cleansed sales/despatch history by item, site, channel, customer, with outlier and promotion adjustment
- Statistical forecasting: baseline models (moving average, exponential smoothing, Holt-Winters seasonal), model selection by item behaviour, forecast horizon and bucketing (day/week/month)
- Forecast enrichment: promotional uplift, new product introduction profiles (analogue items), phase-out curves, marketing/commercial input capture
- Consensus process: forecast versions, collaborative override with reason capture, sign-off by period
- Forecast accuracy measurement: MAPE/bias by item, category, site, planner; accuracy trend driving model reselection

7.4.2 Inventory policy and replenishment

- Policy calculation: safety stock from service level and demand/lead-time variability, reorder point, order-up-to level, EOQ with MOQ/pack rounding
- Policy by segmentation: ABC/XYZ-driven service levels, cold-chain and short-shelf-life special handling
- Replenishment engine: net requirement calculation, suggested purchase and transfer orders, respect for supplier calendars, lead times and site calendars
- Exception-driven planning: stockout risk, excess and expiry risk, policy-breach alerts, forecast-vs-actual divergence

7.4.3 Distribution requirements planning

- Multi-site netting: demand at each site sourced from supplier or from another site, sourcing rules and priorities
- Inter-site transfer suggestions, including expiry-driven redistribution (move short-dated stock where it will sell)
- In-transit visibility and expected-arrival accuracy

7.4.4 Material requirements planning

- Requirements explosion: from forecast and firm demand through BOM levels to component requirements
- Time-phased planning with lead-time offsetting, lot-sizing rules, planning horizons and time fences
- Planned order generation (purchase and works orders), firming, and release to Procurement/Manufacturing
- Pegging: requirement traceable to its source demand; what-if regeneration
- Capacity awareness: rough-cut capacity check against production resources (finite scheduling remains out of scope)

7.4.5 Supply and operations review

- Supply plan vs demand plan reconciliation, gap identification, scenario comparison
- Service level and fill rate reporting, stockout root-cause analysis

- Planner workbench: exceptions first, with drill-through to items, orders and batches

Owns: forecasts and versions, planning policies, planned orders, planning exceptions
 Config: models and parameters, service levels, lot-sizing, time fences, sourcing rules, segmentation rules
 Events: forecast.published, plan.run.completed, replenishment.suggested, planning.exception.raised

7.5 Manufacturing

Purpose: production and assembly control with regulated batch records, from assembly, kitting and repackaging through to process manufacturing.

7.5.1 Product structures and routings

- BOM management: multi-level, versioned, effective-dated, with scrap/yield factors, alternates and substitutes, phantom levels
- Formulations (process mode): quantities by percentage or absolute, active-ingredient potency adjustment, overage rules
- Routings: operations, work centres/resources, setup and run times, labour and machine rates
- Engineering change control: change request → impact analysis (open orders, stock, cost) → approval → effectivity, with full history

7.5.2 Works orders

- Order creation: from MRP, from demand, or manual; order types (production, assembly, kitting, rework, repack)
- Material availability check with shortage detail before release; hard allocation on release
- Component issue: backflush, manual issue, or scan-confirmed issue; batch selection under FEFO with potency/shelf-life rules
- Production execution: operation progress, quantity completed/scrapped with reason, labour and machine time capture, in-process holds
- Receipt of finished goods: batch creation with derived expiry, quarantine routing, label printing trigger, container/pallet identity
- Order closure: variance calculation (material usage, yield, labour, scrap), reconciliation of issued vs consumed vs produced

7.5.3 Batch records and compliance

- Electronic batch record assembled from execution events: materials and batches consumed, quantities, operators, timestamps, equipment, deviations
- Deviation and non-conformance capture with investigation workflow
- qualified release step on the finished batch, with electronic signature capture and audit
- Full forward and backward genealogy across manufacturing (component batch → finished batch → despatches)

7.5.4 Production costing

- Standard cost roll-up through BOM and routing; cost version management
- Actual cost capture: materials at issue cost, labour, overhead absorption
- Variance analysis: material price and usage, labour rate and efficiency, yield, overhead absorption — by order and by period

Owns: BOMs, routings, works orders, batch records, production variances
 Config: order types, backflush rules, yield tolerances, ECO approval chains, costing versions
 Events: works_order.released/completed/closed, component.issued, finished_goods.received, deviation.raised, batch.released_by_authority

7.6 Sales & Order Management

Purpose: every demand channel — automated intake from an upstream system, B2B wholesale, intercompany — through one order lifecycle.

7.6.1 Order capture

- Automated intake channel: order intake from the upstream system with validation, duplicate detection, hold status handling, cancellation cut-off enforcement as a configured, testable rule (not embedded logic with blind spots)
- B2B channel: order entry, customer PO reference capture, quotation → order conversion, order acknowledgement issue, EDI intake later
- Intercompany: transfer-driven order creation with transfer pricing
- Pricing: price lists by customer/channel with validity dates, discount structures, promotions, contract prices, minimum margin checks
- Credit control: credit limit and status check at capture, hold/release workflow with approval, overdue-balance blocking

7.6.2 Order lifecycle

- Configured state machine per channel: statuses, allowed transitions, blocking conditions, cut-off points, all defined as data with transition validation and no unreachable states
- Availability: ATP check at capture, back-order creation, promise-date calculation from stock and inbound supply
- Allocation trigger and re-allocation on supply arrival
- Amendment and cancellation rules by status, with the cut-off consequences made explicit (what can still be recalled, what cannot) and enforced consistently
- Order holds: clinical, payment, credit, stock, address — with release workflow and audit

7.6.3 Sales order document cycle (quote to invoice)

- Quotation: creation, validity dates, versioning, margin visibility, approval on discount thresholds, conversion to order with full traceability

- Sales order: header (sold-to, bill-to, ship-to, customer order reference, payment terms, currency, tax rule, requested date) and lines (item, quantity, UoM, price, discount, promise date, site, delivery split)
- Order types: standard, call-off against a blanket agreement, consignment, sample/free-of-charge, replacement, intercompany, direct/drop-ship
- Order acknowledgement issue and re-issue on amendment
- Delivery documents: delivery/shipment creation from the order (part-deliveries and consolidations supported), picking ticket, delivery note; delivery validation as the controlled point that reduces stock, role-gated so warehouse validates and Finance invoices — never the same permission
- Invoice creation strictly from a validated delivery (enforced structurally, not by convention), carrying quantities, values and the customer order reference through to the printed document
- Invoice types: standard, consolidated (multiple deliveries, periodic billing), pro-forma, credit and debit notes, intercompany
- Handoff to Finance: invoice posting with revenue and COGS recognition, AR open item creation, settlement matching for prepaid channels
- Full document lineage: quote → order → allocation → delivery → invoice → payment, navigable in both directions, each hop evented

7.6.4 Fulfilment coordination

- Release to warehouse: release rules by site, packing line, priority, cut-off time, sequencing (sequencing handled as configuration)
- Document generation triggers: picking ticket, delivery note, labels, customs paperwork
- Despatch confirmation and courier tracking capture; delivery exception handling (failed delivery, return to sender)
- Customer notification events back to the originating channel
- Open order book management: unfulfilled order ageing, back-order release on supply, order-to-despatch cycle time measurement

7.6.5 Returns, credits and complaints

- Return authorisation with reason coding, return receipt and disposition
- Credit note request workflow with approval; link to original order and invoice
- Complaint capture with regulatory routing (adverse event, product quality complaint) — evidence trail to batch

7.6.6 Global and detailed allocation

The behaviour to model, stated precisely: global allocation checks whether stock exists at the site and reserves quantity without committing to specific stock; detailed allocation commits to specific stock lines — and in some operations it must find that stock within a named area before the order can be processed. The gap between the two is where orders sit stuck.

A familiar failure class: an order that is globally allocated but cannot be detail-allocated because the stock, while present at site, is not in the named operational area — visible as a shortage even though the site holds stock.

TWO-STAGE ALLOCATION, MODELLED EXPLICITLY

- Stage 1 — global allocation: reserves quantity against site-level availability. No batch, no location. Confirms the order can be promised and protects the quantity from other demand. Fast, cheap, reversible
- Stage 2 — detailed allocation: commits specific stock lines — batch, location, container, status — under the applicable policy. This is what a picker acts on
- Both stages are first-class allocation records with their own lifecycle, so an order's position is always unambiguous: promised, committed, or blocked, and precisely why

LOCATION SCOPE AS POLICY, NOT HARD-CODING

- Detailed allocation runs against a configurable location scope: a named operational area, pick faces only, whole site, or a named zone set — configured per channel, product category, site, and order type
- The named operational area constraint becomes a scope rule, so a new site or a changed layout is configuration
- Combined with batch policy: FEFO ordering, minimum remaining shelf life, released-status-only, single-batch-per-order, container preference — all evaluated at stage 2

CLOSING THE GAP BETWEEN THE STAGES

- When global succeeds and detailed fails, the system does not simply mark the order short. It classifies the cause: stock present but outside scope, quarantined, expiry-blocked, allocated elsewhere, or genuinely absent
- Stock present but outside scope raises an internal replenishment requirement — move stock into the named operational area — which flows to the warehouse as a directed task rather than sitting as an order exception nobody owns
- Auto-retry on replenishment: the order detail-allocates the moment qualifying stock arrives in scope, with no manual re-run
- Exception queue by cause, with ageing and thresholds, so the pattern is visible before it becomes a backlog

ALLOCATION INTEGRITY

- Reallocation rules: priority order (channel, customer, date, clinical urgency), permitted reallocation between orders with approval and audit
- Reservation ageing: unconsumed detailed allocations expire back to available after a configured period, preventing silent stock lock-up
- Overrides: manual batch or location selection with reason and role permission, always audited
- Counting interaction : allocated and detail-allocated stock is excluded from count scope by default, so counting and allocation stop fighting each other

Configuration: allocation stages required by channel and order type, location scopes, batch selection policies, shelf-life minimums, priority rules, reservation ageing, override permissions, replenishment trigger thresholds Events: allocation.global.made/released, allocation.detailed.made/released/failed,

Owns: orders across channels, allocations link, holds, returns, credits requests
Config: state machines, cut-off rules, release and sequencing rules, price lists, credit rules, hold types
Events: order.received/validated/allocated/released/picked/despatched/cancelled, hold.applied/released, return.authorised

7.7 Finance

Purpose: full accounting integrity — from operational events to a defensible ledger. This is the last domain to move to Own, but its model must exist from the start because every operational event carries accounting consequence.

7.7.1 Chart of accounts and ledger structure

- Chart of accounts: account code structure, account types (asset, liability, equity, income, expense), control accounts, account groups, active/blocked status, per-entity variation with group mapping
- Multiple ledgers: statutory/legal ledger, group/IFRS ledger, tax ledger, management ledger — parallel postings from the same event with different rules where required
- Fiscal calendar: financial years, periods, period status (open, closed, permanently closed) by ledger and entity
- Journals: journal types, numbering, source coding (which subsystem produced it), manual journal entry with approval, recurring and reversing journals, templates
- Posting engine: operational event → accounting rule → journal lines; rules defined as configuration by event type, product category, site, entity (so a class of goods cannot silently post to the wrong account — the mapping is visible, testable and owned)

7.7.2 Analytical dimensions

- Dimension types: cost centre, department, brand, channel, product category, site, project, customer group, entity-defined others
- Dimension rules: which dimensions are mandatory per account, default derivation from the source event, validation at posting, dimension combinations allowed/blocked
- Dimensional reporting: P&L and balance by any dimension combination, allocation rules for shared costs
- Statistical/analytical ledger for non-financial measures alongside values

7.7.3 Subledgers

- Accounts payable: supplier invoice capture (with match to PO/GRN), approval workflow for non-PO spend, payment proposal and run, remittance, ageing, supplier statement reconciliation, accruals from GRNI
- Accounts receivable: customer invoicing from validated delivery, settlement/PSP reconciliation for prepaid channels, cash application and matching, ageing, dunning workflow, bad debt provisioning
- Inventory accounting: stock valuation postings from movement events, GRNI accrual, purchase price variance, revaluation, provision for obsolete and short-dated stock, cost of goods sold recognition on despatch
- Fixed assets: register, categories, depreciation methods and schedules, additions, disposals, transfers, impairment, capital work in progress

7.7.4 Tax

- Tax determination: tax codes by entity, jurisdiction, item, party, transaction type; multi-jurisdiction rules (UK VAT, EU, US sales tax, Canada GST/HST/PST)
- Tax reporting: VAT return preparation, MTD-compatible submission handling, EC sales lists, Intrastat where applicable, reverse charge handling
- Tax audit trail: every posting traceable to its determination rule

7.7.5 Intercompany and consolidation

- Intercompany trading: matched postings both sides from one transfer event, transfer pricing rules, intercompany balance reconciliation, elimination identification
- Multi-currency: transaction, entity, and group currency; rate types and tables; revaluation of monetary balances; translation for consolidation
- Consolidation: entity roll-up, elimination entries, minority interests where relevant, group reporting pack

7.7.6 Period close

- Close calendar with tasks, owners, dependencies and status; automated task completion where the system can prove it
- Automated close checks: subledger-to-GL reconciliation, stock ledger-to-GL agreement, GRNI reconciliation, suspense account clearance, unposted document sweep, intercompany balance match, consolidation health — consolidation defects become blocking close checks rather than later discoveries
- Accruals and prepayments, provisions, cut-off testing
- Close evidence pack: every check's result, every reconciliation, every manual journal with approval — assembled automatically for audit

7.7.7 Financial reporting

- Statutory: trial balance, P&L, balance sheet, cash flow, by entity and consolidated
- Management: by dimension, period comparatives, budget vs actual, variance commentary support
- Drill-down: any reported figure → journal → source document → source operational event → the physical movement. End to end, always.

Owns: chart of accounts, ledgers, journals, dimensions, subledger balances, tax determination, close state. Config: COA, ledger definitions, posting rules by event type, dimension rules, tax rules, close checklist, approval thresholds Events: journal.posted, period.closed, reconciliation.completed, tax.return.prepared

7.8 Quality, Compliance and Traceability

Purpose: make regulated obligations a property of the data rather than a project each time they are tested.

- Quality management: inspection plans by item/supplier, sampling rules, test results capture, certificate of analysis storage, disposition (accept, reject, concession), supplier quality notifications
- Quarantine control: automatic quarantine on receipt by rule, release only by an authorised qualified role, blocked stock handling, quarantine ageing
- Deviation and CAPA: non-conformance capture, investigation, root cause, corrective and preventive actions with owners and due dates, effectiveness review
- Recall management: batch → full downstream despatch list (customer, order, quantity, date) generated on demand; recall workspace with action log, customer contact tracking, quantity reconciliation (recovered/destroyed/unaccounted), evidence pack export; continuous recall-readiness measurement against configured regulatory clocks
- Temperature and condition excursions: capture, impact assessment against affected batches, disposition
- Regulatory reporting: licence-linked reporting obligations, controlled drug registers where applicable
- Audit support: full audit trail export by object, period or batch; access history; validation evidence for computerised system compliance

Config: inspection plans, quarantine rules, release authorities, recall templates, CAPA workflows Events: inspection.completed, batch.quarantined/released, deviation.raised, recall.initiated/closed

7.9 Logistics and Transport

- Shipment planning: consolidation, load building, route and carrier selection by cost/service rules
 - Carrier integration: label generation, tracking number capture, status updates, proof of delivery
 - Freight cost: rate cards, expected vs actual freight, cost allocation to landed cost
 - Customs and cross-border: commodity codes, country of origin, export/import documentation, duty estimation, intercompany cross-border compliance
 - Delivery performance: OTIF by carrier and site, failed-delivery analysis
-

7.10 Cross-cutting operational modules

7.10.1 Integrity & Reconciliation

- Continuous automated checks across every domain (financial, stock, order, master data), threshold-driven alerting, run history with row-level evidence, close-blocking check designation, seeded-defect testing

7.10.2 Reporting, Dashboarding & Analytics

- Report catalogue: governed, versioned reports per domain (stock movement and valuation, order book and cycle time, purchasing and GRNI, planning exceptions, production variance, financial statements and dimensional P&L, quality and recall evidence)
- Self-serve access: role- and site-scoped, parameterised by date/site/entity/dimension, on-screen with drill-down, export to CSV/XLSX/PDF
- Dashboards by audience: executive (cross-entity health, KPIs, go-live status), finance (close progress, reconciliation states, ageing, GRNI), planner (forecast accuracy, cover, exceptions), warehouse and site (throughput, accuracy, open orders, print queues), ERP team (integration health, job runs, error queues)
- KPI framework: definitions owned centrally with a single calculation per metric, targets and thresholds, trend and period comparatives — so a number means the same thing in every report
- Scheduled distribution: subscriptions by user/role, figure packs assembled automatically (weekly update pack, month-end pack, MBR pack), delivery to email and Slack
- Ad-hoc analysis: governed views exposed for analyst querying and BI tools, with natural-language read-only querying over the same views
- Report governance: every figure traceable to its source event and document; report versions and changes audited; deprecation path so shadow spreadsheets can be retired against a named replacement

7.10.3 Document & Print Services

- Template registry (labels, picking tickets, delivery notes, invoices, customs docs) with versioning, preview and environment promotion; print queue orchestration and health monitoring; reprint with audit; output archiving with retrieval by document reference

7.10.4 Workflow & Approvals (platform capability)

- Reusable approval engine consumed by every module: chain definition by entity/site/value/category, sequential/parallel/conditional steps, delegation, escalation on ageing, mobile-friendly approval, complete approval audit, re-approval rules on change

7.10.5 System Administration

- User and role administration with request → approval → provision workflow, periodic access review packs, segregation-of-duties rule checking, scheduled job management with evidence, housekeeping automation, environment management, configuration change history and promotion

7.11 Functional coverage summary

- Covered end to end: master data, inventory and warehouse (including counting, FEFO, lot and expiry), procurement with approval workflow, supply chain planning (demand, forecasting, replenishment, DRP, MRP), manufacturing with batch records, sales and order management across channels, finance (COA, ledgers, dimensions, subledgers, tax, intercompany, close, reporting), quality and traceability, logistics, and the cross-cutting integrity, insight, print, workflow and admin layers
- Deliberately out of scope: HR and payroll, CRM and marketing automation, clinical/prescribing systems (remain external), e-commerce storefront, finite-capacity scheduling, treasury and hedging
- Delivery sequencing, phasing and the strangler cutover order remain as set out in the Product Document; this specification defines the destination, not the order of travel

7.12 Approval routing

The behaviour to preserve

Users belong to a department. Each department has one or two approvers selected by the value of the request: below a threshold it routes to the first, above it escalates to a higher authority. That pattern survives unchanged and becomes configuration rather than convention.

New objects

- `department` — organisational unit within an entity: code, label, manager, optional parent for hierarchy, optional default cost centre and dimension set
- `principal_department` — membership, effective-dated, one primary and optional secondaries
- `approval_band` — per department and object type: lower and upper value bound, currency, approver resolution, sequential or parallel, whether lower bands re-run when a higher band applies
- `approver_assignment` — subject (user, user group, or department), object type, assigned approver, effective dates, optional value bounds, mode (*replaces* or *prepends*)

A document inherits the requester's primary department at capture, and that inherited value routes it — not the department the requester belongs to at approval time. The routing decision is stamped on the document.

Two modes, both live at once

Department and threshold driven — the default. An ordered set of value bands per department: band 1 up to a configured amount to one approver, band 2 above it adding a higher authority sequentially, further bands as needed. Approver resolution within a band, in precedence order: a named user, a role within the department, a role at entity or site level, or the requester's line manager resolved from the hierarchy.

Named approver assignment — the addition. Specific users or groups route to a designated approver regardless of department: a contractor whose requests go to their engaging manager, a new starter under supervision, a project team whose spend goes to the project owner.

Resolution order: named assignment, then department band, then entity fallback. Every resolution records which rule selected each approver and the version of that rule.

Controls applying to both

- Self-approval refused: where resolution would produce it, escalate to the next authority
- Delegation and substitution time-bounded and recorded; the approver of record is the assigning user with the delegate named
- Escalation on ageing per band, using the site calendar
- Re-approval on material change with configurable tolerance, restarting from the band the new value falls into
- Vacancy handling: a band resolving to nobody holds the request and raises an exception rather than routing silently upward
- Multi-currency bands resolve on the rate in force at raise date

Configuration: departments and hierarchy, memberships, bands per department and object type, approver resolution order, named assignments, tolerances, escalation timers, vacancy behaviour Events: `approval.chain.resolved`, `approval.requested`, `approval.granted`, `approval.rejected`, `approval.escalated`, `approval.delegated`, `approval.reapproval.triggered`

7.13 Product master completeness

Confirmed as already specified in §1.1: product category and family, `item_site` per-site parameters (reorder policy, min/max/safety stock, lead time, count class, default locations, planning flags, valuation site), standard cost per item and entity with costing versions, and the full UoM set with conversions.

The addition: default supplier

§3.1 carries the supplier-item catalogue but nothing states which supplier is preferred when a requirement is generated.

- `item_supplier` — the catalogue link extended with preference rank, default flag per item and site, sourcing split percentage where a requirement is deliberately shared, approved-for-use flag, effective dates
- Replenishment, MRP and manual purchase order creation take the default for that item at that site; override permitted and audited

- A regulated item's default supplier must also sit on the approved supplier list for that product class, or selection is refused
 - Supplier changes are effective-dated, so historical orders retain the supplier that was default at the time
-

7.14 Analytical dimensions

Specified in §7.2, confirmed here with one change. Department, project and location are all available, alongside cost centre, brand, channel, product category, site, customer group and tenant-defined axes.

The change: **the department object in §1 is the same object used as the finance dimension**. Defining an approval department and a dimension department separately is how reporting ends up disagreeing with operations.

The rules that make dimensions load-bearing rather than decorative: - Mandatory-by-account: which dimensions must be present for a posting to a given account, enforced at posting - Derivation from the source event: department from the requisition, location from the movement's site or bin, project from the document header — operators are not asked to key what the system knows - Permitted combinations: which values may appear together, which are blocked - Project dimensions carry validity dates, so a closed project stops accepting postings

7.15 Marshalling-area depletion

The behaviour requested

When a sales order is planned and released to a printer, the stock it consumes is held in a designated area and depletes from there — not from anywhere in the warehouse.

This closes §6.6 from the other side: that section handles what happens when stock is out of scope, this specifies how it gets into scope.

The marshalling area as a configured stage

- A marshalling area is a location or zone type — staging, pick face, dispensary, or a named zone set — configured per site, channel, product category and order type
- Detailed allocation at release commits against the marshalling area scope only, so the picker is directed to stock that is physically where they will collect it
- Stock reaches it by an explicit recorded movement from bulk, with the container hierarchy updating if a pallet is broken down

Two replenishment modes, selectable per product class

- **Pull, wave-driven:** on release, compute the requirement against the marshalling area, find the shortfall, raise directed replenishment tasks from bulk. Picking waits on a visible task rather than on a stuck order
- **Push, policy-driven:** min and max levels at the marshalling area per item and site, topped up continuously or on schedule regardless of the day's orders — suited to fast movers

Both can run for different product classes at the same site.

Depletion and integrity

- Picking consumes from the marshalling area; the movement records bin, batch and container, so bulk and marshalling-area on-hand are separately visible and separately accurate
- Marshalling-area stock is allocated stock: excluded from count scope by default, unavailable to other demand, and aged so stock staged but not picked returns to bulk after a configured period
- Where the marshalling area holds stock the order cannot use — wrong batch, insufficient shelf life, quarantined — detailed allocation classifies the cause per §6.6 and raises replenishment rather than reporting a shortage
- **Printing is downstream of successful detailed allocation, not concurrent with it.** Documents and labels are produced only for orders with committed stock in the marshalling area, which structurally removes the failure mode where labels exist for an order that cannot be picked

Configuration: marshalling area per site and channel, pull or push per product class, min and max levels, replenishment thresholds, staging ageing and return-to-bulk period, print trigger point relative to allocation Events: `release.wave.opened`, `replenishment.task.raised`, `replenishment.task.completed`, `stock.movement.recorded`, `allocation.detailed.made`, `document.printed`

7.16 Account determination

The mechanism

§7.1 states that operational events post through declarative rules. What it never named is how an item carries its accounting identity.

The item carries a **accounting code, not a GL code**. An item holding an account directly must be re-coded per entity, per ledger, per transaction type. An item holding a class is coded once and resolved by rule.

- `item_posting_class` — tenant-configured, accounting-only classification: finished good, raw material, packaging, consumable, non-stock service, sample, spare part, asset-held-as-stock, and whatever else the organisation needs. Every item carries exactly one, mandatory, validated at creation

- `party_posting_class` — domestic supplier, EU supplier, rest-of-world, intercompany, staff — because the payable or receivable account often depends on the counterparty as much as the item

The determination matrix

An account resolves from a combination, not a single field: - Transaction type — receipt, issue, despatch, adjustment, revaluation, scrap, production issue, production receipt, invoice, credit, write-off - Item accounting code, and party accounting code where a party is involved - Site and entity - Ledger — statutory, group, tax and management may resolve differently from the same event - Legislation pack in force - Movement reason code where one exists — a write-off for damage and one for expiry can post differently

The rule returns the account **and** the dimension set together, because a posting without its dimensions is only half determined.

Illustrative shape: - Receipt, *finished good*, UK entity → finished goods stock debit, GRNI accrual credit - Receipt, *consumable*, UK entity → consumables expense debit, GRNI accrual credit, department dimension derived from the requisition so consumable spend reports by department without anyone keying it - Despatch, *finished good* → cost of goods sold debit, finished goods stock credit - Scrap, *finished good*, reason *expiry* → expiry write-off rather than general scrap

Rules that keep it honest

- Every determination records which rule, which version, which inputs produced the account
- **No default-to-suspense.** An unmatched posting is refused and raises an exception. A silent fallback to suspense is how packaging ends up in cost of sales and nobody notices for a quarter
- **Coverage assertion:** for every combination of accounting code, transaction type and entity that can occur, a rule must exist. Testable, and belongs in the conformance suite — run before promotion, not after month-end
- Changing an item's accounting code is effective-dated and approval-gated, never retrospective
- Document-level overrides permitted where genuinely needed, but explicit, reasoned, permission-gated and audited

Configuration: accounting codes for items and parties, the determination matrix per entity and ledger, reason-code mappings, override permissions Events:

`posting.rule.resolved`, `posting.determination.failed`, `item.posting_class.changed`

7.17 Product classification and code composition

A caution worth taking seriously

Codes that encode meaning become wrong. A product changes pack size, gets rebranded, moves category — and the code, referenced by every historical document, label, batch record and customer purchase order, now says something false. Correcting it breaks the audit trail; leaving it means living with a code that lies.

So the model separates three things: - **Identity is opaque and permanent** — a surrogate key, meaningless, never reused - **Classification is structured data** — brand, form, strength, pack size, therapeutic class, each from a controlled vocabulary. This is what reporting, filtering, search and pricing actually use - **The composed code is a derived label** — generated from the classification, human-readable, unique, stable thereafter. A rendering of the attributes, not the truth of them

The organisation keeps the readable code its people rely on, without making it load-bearing.

Classification model

- `classification_axis` — tenant-defined axis: brand, product type, dosage form, strength, pack size, therapeutic class, hazard class, storage condition. Code, label through the resource layer, position
- `classification_value` — permitted values per axis: code, label, abbreviation for code composition, effective dates, optional parent for hierarchical axes
- `item_classification` — values assigned to an item, one per single-valued axis, several where multi-valued
- Which axes are mandatory for which product category is configuration: a medicinal product may require brand, form, strength and pack; a packaging component only material and size
- Values are controlled vocabularies. Free text is not a classification

Code composition templates

- `code_template` — named, versioned pattern of ordered segments; each segment draws from a classification axis abbreviation, a fixed literal, a sequence, or a check character
- Per segment: source, length, padding, casing, required or optional
- Bound per product category and per entity, so product families can code differently within one tenant
- Illustrative: `[type:2][brand:3][strength:4][pack:3][sequence:4]`
- Sequence segments draw from the numbering service, so uniqueness holds under concurrency
- Check characters optional, for codes read aloud or keyed by hand

Generation and control

- On creation the classification is captured first, the code proposed from the template, visible before commit, editable only where the template permits manual segments
- Uniqueness enforced on the composed code as well as the surrogate key
- Templates versioned and effective-dated: changing the scheme does not re-code existing items, and each item records which template version produced its code

- Legacy and external codes live in `external_ref` — a migrated code, a customer's part number and a supplier's catalogue reference coexist without competing to be the real one
- Where an attribute later changes and the code no longer reflects it, the platform flags the divergence. The organisation decides: leave it, or supersede the item. Doing it by accident is not an option

Connections

- Classification axes feed the \$7.2 dimensions from the same controlled vocabularies, so a brand means the same thing in a P&L as in a stock report
- Posting class stays deliberately separate from commercial classification: accounting behaviour is not derived from brand or size, because the two change for different reasons and on different timescales
- Labels resolve through \$2.10, so an axis named differently per locale is one object with several renderings

7.18 AI-assisted setup of these surfaces

Everything above is exactly the kind of configuration \$2.8 exists to author. The onboarding assistant should be able to produce all six as reviewable diffs.

What the assistant proposes

- **Organisation and approvals:** interview the administrator about departments, who manages them, and the amounts at which approval escalates — produce the department hierarchy, memberships, and banded chains
- **Posting classes and determination:** given the chart of accounts and the product categories in use, propose accounting codes and a determination matrix, and report which combinations have no rule
- **Classification and code template:** given a sample of existing product codes, infer the axes and segment structure, propose the vocabularies and a template, and flag codes that do not fit the inferred pattern
- **Item defaults:** propose per-site planning parameters and default suppliers from purchase history
- **Marshalling areas:** propose zones and replenishment mode per product class from movement history
- **Migration mapping:** map an incoming file to items, classifications, accounting codes and departments, flagging anomalies and gaps

Guardrails, unchanged from \$2.8

- Proposes, never applies. Every output is a diff requiring named human approval, with proposer and approver both recorded
- Validated in test against the conformance suite before promotion
- Never in the transaction path: the assistant authors the determination matrix, it never determines an individual posting
- Explainability: a proposal cites the data and precedents it drew on — “these 14 product codes share a three-character second segment matching your brand list”
- Reversible: every applied change is versioned and revertible with a recorded reason

Honest status

\$2.8 specifies this and the proposal pipeline exists behind the AI boundary. The onboarding interview itself — the conversational surface that turns an administrator's answers into a candidate configuration — is not among the current routes. It is the natural next build once these six surfaces exist, because an assistant with nothing to configure has nothing to propose.

PART 8 — EXTENSION MODEL

How an organisation's specific way of working is expressed without touching the product.

8.1 The extension ladder

- Level 1 — Configuration: policies, rules, thresholds, chains, state machines, terminology, templates, report definitions. Covers the overwhelming majority of organisational difference. No engineering, no deployment
- Level 2 — Declarative extension: tenant-defined fields on standard objects, tenant-defined validation rules, tenant-defined document types bound to existing behaviour, tenant-defined events and subscriptions. Configured, versioned, promotable, still no code
- Level 3 — Integration: adapters to the organisation's other systems, defined by connection, mapping and schedule; inbound events and outbound commands through the standard gateway
- Level 4 — Product change: genuine capability gaps become product backlog, built generically and shipped to every tenant. A tenant-specific build is never the answer

8.2 Rules that keep the product reusable

- No branch in application code may test for a tenant, a customer name, a site name or a jurisdiction
- No tenant's configuration is referenced by, or required for, another tenant's operation
- No product default is derived from a customer artefact; defaults come from standards, legislation packs or neutral templates

- Every capability added for one organisation must be expressible as configuration for others, or it does not enter the product
- A tenant may hold nothing that the product cannot describe generically: if a concept has no home in the canonical model, the model is extended, generically, for everyone

8.3 Deployment profiles

- Each customer implementation is documented as a deployment profile held within that tenant: organisational structure, legislation and language selections, policies chosen, process variants, integration inventory, migration mappings, acceptance fixtures
- Deployment profiles are tenant content. They are never distributed with the product and never referenced by product documentation
- The product's own documentation uses neutral illustrative examples only

PART 9 — OPERATIONS, ASSURANCE AND LIFECYCLE

9.1 Observability, distinct from audit

Audit answers who did what, for regulators. It does not answer whether the platform is healthy.

- Distributed tracing on the correlation identifier already carried by every event
- Service and job metrics, queue depth, consumer lag, projection lag as first-class signals
- Health endpoints per service, and service-level objectives with alerting
- A job that stops being scheduled raises an alert — a silent job is the failure mode that looks like success

9.2 Non-functional commitments

A warehouse cannot pick when the platform is down, so these are stated rather than assumed.

- Availability target and maintenance windows, published per deployment
- Recovery point and recovery time objectives, with backup verification by periodic restore drill rather than by backup success logs
- Performance budgets on the hot paths: allocation, receipt confirmation, despatch scan, document transition
- A capacity model per tenant, with headroom monitoring

9.3 Validation and electronic signature

The target profile includes regulated manufacture and distribution, where a system cannot be deployed without validation evidence.

- Validation posture shipped as product content: validation master plan template, installation, operational and performance qualification templates, and a requirement-to-test traceability matrix generated from the conformance suites
- Electronic signatures meeting the common regulatory expectations: signature meaning captured explicitly, two components, non-repudiation, manifestation on the signed record, and tooling for periodic audit-trail review
- Which actions require signature is configuration per tenant and legislation pack, not a code decision

9.4 Personal data in an immutable store

Tenant deletion is handled by key destruction. Erasure of an individual within a live tenant is in direct tension with an append-only store, so the position is stated rather than discovered.

- Events carry references, never personal data. Personal data lives in a separately keyed, mutable store that can be erased without touching the ledger
- Where a jurisdiction requires erasure, the reference remains and the referent is destroyed, leaving the ledger arithmetically intact and the individual unidentifiable

9.5 Product upgrade across tenants

- Whether tenants may sit on different product versions is a stated policy, not an accident of deployment
- Tenant configuration is migrated forward when product content changes, with backward-compatibility guarantees for configuration objects and a declared deprecation window
- A tenant's own acceptance fixtures run against the new version before it lands for that tenant

9.6 Coexistence, migration and cutover

Design objective O5 promises running alongside an incumbent and assuming domains progressively. That requires machinery, not only intent.

- A reconciliation framework: parallel-run comparison per domain, divergence classification, tolerance and ageing
- Opening balance loading with as-at dating, so history begins honestly
- Per-domain cutover state per tenant, with a defined rollback and a dual-read period
- Cutover is evidence-gated: a domain moves when its reconciliation has been clean for a declared period, not when a date arrives

9.7 The tenant-facing contract

- Versioned API with a published deprecation policy
 - Webhooks with subscription management and replay
 - Rate limits and quotas per tenant, published
 - A sandbox environment per tenant, seeded from starter packs rather than from live data
-

PART 10 — WHAT THE PRODUCT REFUSES

- No unscoped data: every row, event, file and job carries a tenant
 - No tenant-specific code paths, ever
 - No customer-derived content in product artefacts, defaults, templates, documentation or model training
 - No hard-coded jurisdiction behaviour; legislation is packs
 - No hard-coded user-facing text; every string is a resource
 - No mutable operational balances; state is derived from events
 - No direct writes to external systems outside the gateway
 - No configuration edits directly in a live environment; promotion is the only route
 - No unversioned rule, unaudited action or un-replayable integration
 - No artificial intelligence in the transaction path, and no configuration applied without human approval
-

PART 11 — BUILD SEQUENCE

- B1 Tenancy and identity: tenant model, organisational hierarchy, authentication, roles, row-level security, isolation test suite
- B2 Audit and event store: append-only streams, envelopes, outbox, cursors, replay
- B3 Configuration engine and rule engine: typed versioned effective-dated objects, declarative evaluation, rule tracing
- B4 State machine engine and workflow: lifecycle definitions with transition validation; the shared approval engine
- B5 Localisation: legislation pack structure and binding, conformance harness, resource and terminology layer
- B6 Promotion and rollback: change sets, diff and preview, gated promotion, snapshots, drift detection
- B7 Canonical domain model: organisation, parties, products, stock ledger, document spine, planning, finance structures, quality, integration registry — with invariant tests as first-class artefacts
- B8 Integration foundation: adapter contract, write gateway with idempotency and approvals, message log, monitoring and replay
- B9 Scheduler, notifications, reporting substrate and governed views
- B10 Configuration intelligence, on top of a mature configuration engine
- B11 Starter content packs and the capability registry (Part 11), so a tenant is onboarded from packs rather than by hand
- Only then: functional modules, in an order set by the first deployment's needs — built generically, configured per tenant

The foundation is finished when a second organisation can be onboarded, configured and operated by someone who has never seen the first.

PART 12 — STARTER CONTENT PACKS

Neutral defaults, switchable, so a new organisation can begin operating on day one. Product content in the sense of Part 2.3: shipped with the release, identical for every tenant, applied at onboarding as a change set, and copied into the tenant rather than referenced.

12.1 The governing rules

- **Neutral by construction.** Every value derives from a published standard, a statutory structure or generic commercial practice. None derives from any tenant's configuration
 - **Copied, not referenced.** Applying a pack writes a copy into the tenant, which then owns it. The product template is untouched thereafter
 - **Applied through the normal path.** A pack lands as a change set, previewed and promoted, so onboarding leaves the same audit trail as everyday work
 - **Everything is switchable.** No pack, capability or individual object is mandatory
 - **Versioned.** A tenant records which pack versions it started from, so the origin of any value is answerable years later
-

12.2 Capability switches

The answer to “comprehensive but not overwhelming” is not less content. It is content that arrives switched off until wanted.

12.2.1 The capability registry

- **capability** — a named, switchable unit of product behaviour: batch control · expiry control · serialisation · container identity · quality inspection · quarantine and release · multi-entity · intercompany trading · multi-currency · landed cost · consignment stock · third-party custody · contract manufacturing · production · planning and MRP · forecasting · marshalling areas · wave picking · cycle counting · credit control · prepaid settlement · returns · recall management · print orchestration · automated order intake · project accounting · fixed assets
- Each is tenant-scoped, effective-dated, and switched through a change set like any other configuration
- Capabilities declare dependencies: expiry control requires batch control; recall management requires batch control and traceability; contract manufacturing requires ownership separation. Enabling one offers its prerequisites; disabling one warns what depends on it
- A capability gates three things at once — the navigation a user sees, the fields a form shows, and the rules the engine evaluates. Switched off is not merely hidden: its rules do not run

12.2.2 Switching off safely

- A capability with live data cannot be silently disabled. Disabling batch control while batches exist is refused with a named reason, not permitted with a warning
- Disabling is effective-dated and reversible; historical records keep the behaviour in force at their time
- Every switch is audited with actor, reason and change set

12.2.3 Three presets, then adjust

- **Minimal** — master data, purchasing, stock, sales, basic finance. The smallest set that lets an organisation buy, hold and sell
- **Standard** — Minimal plus batch and expiry control, counting, approval routing, quality inspection, landed cost, marshalling areas
- **Full** — Standard plus production, planning and MRP, forecasting, quality management and recall, multi-entity and intercompany, serialisation, container identity, project accounting

A preset is a starting selection, not a tier. Anything can be switched individually afterwards.

12.3 Base pack — organisation and access

12.3.1 Departments

Executive · Finance · Procurement · Supply Chain and Planning · Operations · Warehouse · Production · Quality · Sales · Customer Service · Logistics · Facilities · Information Technology · People

Each with a suggested cost centre code and an empty approval band set.

12.3.2 Role templates

Administrator · Finance manager · Finance clerk · Buyer · Procurement manager · Planner · Production supervisor · Production operator · Warehouse manager · Warehouse operative · Quality manager · Quality inspector · Responsible person · Sales manager · Sales administrator · Customer service · Auditor (read-only) · Integration service account (service account)

12.3.3 Segregation-of-duties rules

Pre-declared conflicts, each switchable: raise and approve a requisition · approve a purchase order and receive against it · validate a delivery and raise its invoice · post a journal and close the period · amend a batch and release it · create a supplier and pay it · adjust stock and approve the adjustment · grant permissions and use them

12.3.4 Approval band skeleton

Three bands per department with thresholds left blank and **promotion refused until they are set**, so the decision happens at onboarding rather than leaving a chain that approves everything silently.

12.3.5 Calendars

Standard working week, two-shift and three-shift patterns, and public holiday sets for the jurisdictions of the bound legislation packs. Sites inherit and override.

12.4 Base pack — master data vocabularies

12.4.1 Units of measure

From UNECE Recommendation 20: count (each, pair, dozen, pack, case, carton, pallet), mass (mg, g, kg, tonne), volume (ml, cl, l, m³), length (mm, cm, m), area (m²), time (minute, hour, day). Within-family conversions pre-populated.

12.4.2 Commercial terms

- **Payment terms** — net 7, 14, 30, 45, 60, 90; end of month; end of month plus 30; prepaid; on delivery; 2/10 net 30
- **Incoterms 2020** — the full eleven, each flagged for where cost and risk transfer, so landed cost and revenue recognition rules can reference them
- **Delivery terms** — standard, express, next day, timed, collection, drop-ship

- **Currencies** — ISO 4217, with rate types: spot, monthly average, period close, budget
- **Countries and regions** — ISO 3166, with customs-union membership flags for tax determination

12.4.3 Item accounting codes

finished good · semi-finished good · raw material · packaging · consumable · spare part · non-stock service · sample · promotional item · asset held as stock · customer-owned material · supplier-owned consignment

12.4.4 Party accounting codes

domestic supplier · EU supplier · rest-of-world supplier · intercompany supplier · domestic customer · EU customer · rest-of-world customer · intercompany customer · carrier · employee · other

12.4.5 Classification axes

Populated where a standard exists: product type · storage condition (ambient, cool, refrigerated, frozen, controlled, hazardous) · hazard class (UN transport classes) · country of origin (ISO 3166) · commodity code structure Empty for the tenant: brand · product family · size or strength · pack format · therapeutic or application class

12.4.6 Code template

One neutral default — `[type:2]-[family:3]-[sequence:5]` — offered rather than imposed.

12.4.7 Location types

receiving · quarantine · quarantine cold · bulk storage · cold storage · frozen storage · controlled storage · pick face · replenishment staging · marshalling area · packing · despatch staging · returns · damages · quarantine returns · production input · production output · in transit · consignment · third-party custody

Each with default attributes: holds available stock, countable, valid allocation scope, default storage condition.

12.4.8 Number sequences

Per document type, pattern `PREFIX-{YYYY}-{0000000}` . Statutory-gapless flags default off unless a bound legislation pack requires otherwise.

12.5 Base pack — states and lifecycles

Version 1 covered document states only. Non-document lifecycles matter just as much and were missing.

12.5.1 Document states

Requisition draft → submitted → in approval → approved → converted; rejected, cancelled **Purchase order** draft → in approval → approved → issued → acknowledged → partially received → received → closed; amended, cancelled **Goods receipt** draft → recorded → inspected → putaway → closed; quarantined, rejected **Quotation** draft → issued → accepted → converted; expired, declined, withdrawn **Sales order** draft → confirmed → held → allocated → released → picked → despatched → invoiced → closed; back-ordered, amended, cancelled **Delivery** draft → picked → packed → validated → despatched; part-delivered, cancelled **Works order** planned → firmed → released → in progress → completed → closed; held, scrapped, cancelled **Transfer order** draft → approved → issued → in transit → received → closed; discrepancy, cancelled **Count** scheduled → in progress → counted → under review → approved → posted; recount, cancelled **Return** requested → authorised → received → inspected → dispositioned → closed; refused **Supplier invoice** received → matched → approved → posted; disputed, rejected

12.5.2 Batch states

created → quarantine → under test → released → in use; blocked · recalled · expired · consumed · rejected · disposed Transitions requiring named authority are flagged: release, unblock, expiry extension.

12.5.3 Container states

created → filled → sealed → in storage → picked → despatched; broken down · returned · damaged

12.5.4 Allocation states

requested → global allocated → detail allocated → picked → consumed; failed · released · expired · overridden Failure causes as a controlled vocabulary: no stock · outside allocation scope · quarantined · expiry blocked · allocated elsewhere · shelf-life insufficient · awaiting replenishment

12.5.5 Planned order states

proposed → reviewed → firmed → released → converted; rejected · superseded · cancelled

12.5.6 Quality states

Inspection raised → sampled → in test → complete; cancelled **Disposition** pending → accepted → accepted under concession → rejected → returned → destroyed **Quality event** raised → investigating → root cause identified → action assigned → verifying → closed; escalated **Recall** initiated → scoped → notifying → recovering → reconciling → closed

12.5.7 Command and integration states

requested → approved → dispatched → confirmed; failed · ambiguous · compensated · abandoned

The ambiguous state exists deliberately: an external write whose outcome is unknown is a distinct condition needing reconciliation, not a failure to retry blindly.

12.6 Base pack — reason codes

Stock adjustment count variance · damage in storage · damage in transit · theft or loss · found stock · system correction · sample taken · quality failure · expiry write-off · weight or measure correction **Scrap and destruction** expired · damaged beyond use · failed inspection · contaminated · recalled · obsolete · customer return unfit **Return to supplier** wrong item · wrong quantity · damaged on arrival · quality rejection · short-dated on receipt · over-delivery · ordered in error **Customer return** damaged in transit · wrong item despatched · wrong quantity · quality complaint · ordered in error · no longer required · delivery refused **Order hold** credit limit · payment pending · stock shortage · address verification · quality hold · compliance check · customer request · duplicate suspected **Order cancellation** customer request · unable to supply · duplicate order · pricing error · credit refusal · past cut-off **Approval rejection** insufficient justification · budget exceeded · incorrect supplier · incorrect coding · duplicate request · requires alternative sourcing **Batch amendment** data entry correction · retest result · supplier documentation update · regulatory requirement · shelf-life reassessment **Allocation override** customer or clinical urgency · batch specified by customer · nearest expiry first · operational instruction · system correction **Price and discount override** contract price · volume agreement · promotional · goodwill · price correction · competitive match **Period reopen** late invoice · audit adjustment · correction of error · consolidation adjustment

12.7 Base pack — policies and tolerances

The values every ERP needs on day one, which otherwise get invented ad hoc by whoever hits the screen first. Conservative defaults, all switchable.

- **Receipt tolerance** — over and under, percentage and absolute, per product class; behaviour on breach: block, warn, or route to approval
 - **Three-way match tolerance** — price, quantity, total; exception queue on breach
 - **Count variance tolerance** — bands by quantity and value, with approval thresholds above each
 - **Shelf-life minimums** — remaining life on receipt, on transfer, on despatch, with customer-specific overrides permitted
 - **Allocation policy** — FEFO default for expiry-controlled classes, FIFO for the rest; single-batch-per-order and nearest-location options
 - **Costing method** — per accounting code: standard for manufactured, average for purchased, FIFO available
 - **Reservation ageing** — how long an unconsumed detailed allocation holds before returning to available
 - **Marshalling-area ageing** — how long staged stock waits before returning to bulk
 - **Credit control** — check at capture, block at limit, tolerance percentage, overdue-days block
 - **Approval escalation timers** — working days per band before escalation
 - **Re-approval tolerance** — the value change that reopens an approved document
 - **Quarantine defaults** — which accounting codes quarantine on receipt, and quarantine ageing thresholds
 - **Backorder policy** — permit, refuse, or partial-ship, by channel
-

12.8 Base pack — finance

12.8.1 Chart of accounts skeleton

1000–1999 non-current assets · 2000–2999 current assets with inventory sub-ranges for raw material, work in progress, finished goods, goods in transit, consignment and customer-owned · 3000–3999 current liabilities including goods-received-not-invoiced and tax control · 4000–4999 non-current liabilities and equity · 5000–5999 revenue by category · 6000–6999 cost of sales including purchase price, usage, yield and freight variances · 7000–7999 operating expenses, dimension-analysed · 8000–8999 other income and expense · 9000–9999 suspense and clearing, each flagged reconciliation-required and close-blocking

Where a bound legislation pack defines a statutory structure, it wins.

12.8.2 Starter determination matrix

Goods receipt by accounting code · consumable receipt with department dimension derived · supplier invoice match with price variance · despatch to cost of sales · sales invoice with tax per legislation · adjustment by reason · scrap by reason with expiry distinguished from general scrap · production issue and receipt with variances · intercompany transfer both sides · prepaid settlement to a clearing account then to receivables

Every rule ships with the dimensions it derives. The coverage assertion runs at the end of pack application and names any combination still unmatched.

12.8.3 Dimensions

Department (the same objects as §3.1, not a second list) · cost centre, one per department · project, empty with validity dating · location, derived from the site registry · channel and brand, empty Mandatory-by-account defaults: department on 6000 and 7000 ranges, location on inventory accounts, project optional.

12.8.4 Period close checklist

Sequenced tasks with dependencies: subledger-to-ledger reconciliation · stock ledger to control account · goods-received-not-invoiced reconciliation · suspense clearance · unposted document sweep · intercompany balance match · accrual and prepayment posting · provision review · variance review · consolidation health · evidence pack assembly Each flagged blocking or advisory, each with an owner role.

12.9 Base pack — operations

12.9.1 Scheduled jobs

Shipped disabled, enabled per tenant: expiry horizon sweep · reservation and staging ageing · count schedule generation · goods-received-not-invoiced ageing · suspense balance check · stock-to-ledger reconciliation · integration backlog check · silent-job detection · approval ageing and escalation · document sequence gap check · tenant purge sweep past grace period

12.9.2 Notification templates

Approval requested · approval escalated · approval overdue · receipt discrepancy · match exception · stock shortage on release · expiry threshold breached · count variance above tolerance · quality event raised · batch released · job failed · integration backlog above threshold · period close task overdue

Each with channel routing, digest option and quiet-hours defaults.

12.9.3 Printer profiles

Common label sizes and resolutions as named profiles, so a new site registers a printer by choosing a profile rather than describing one.

12.9.4 Sender identity checklist

The DNS records an organisation must publish before it can send as its own domain, presented in onboarding as a verified task rather than as documentation.

12.9.5 Document and label output templates

Purchase order · order acknowledgement · goods receipt note · picking ticket · packing note · delivery note · commercial invoice · credit note · pro-forma · transfer note · works order pack · certificate of analysis · pallet label · carton label · bin label · returns label

Neutral layouts, tenant-brandable through the resource and branding layer.

12.9.4 KPI definitions

One calculation per metric, centrally defined: stock accuracy · inventory record accuracy · stock cover days · slow-moving and obsolete value · on-time in-full · order-to-despatch cycle time · fill rate · supplier on-time delivery · supplier quality rejection rate · purchase price variance · goods-received-not-invoiced ageing · forecast accuracy (MAPE and bias) · production yield · scrap rate · recall readiness time · days sales outstanding · approval cycle time

12.9.5 Report catalogue

Stock on hand and valuation · stock movement · expiry horizon · batch genealogy · open order book · order fulfilment performance · purchase order status · goods-received-not-invoiced · match exceptions · supplier performance · planning exceptions · production variance · count accuracy · quality events · recall despatch list · trial balance · dimensional profit and loss · receivables and payables ageing

12.9.6 Base terminology glossary

The keys most commonly overridden, seeded in the base locale so an organisation renames rather than creates: goods-in or receiving · despatch or goods-out · site or location · party or business partner · item or product · batch or lot · requisition or purchase request · works order or production order · marshall area or staging

12.10 Profile packs

Applied over the base, each switchable, each adding only what its capabilities need.

- **Distribution** — buy and sell without transformation; drop-ship and cross-dock document behaviour, consolidation rules
- **Manufacturing** — works order types (production, assembly, kitting, rework, repack), production reason codes, routing and work-centre starters, backflush policy defaults, yield tolerance bands
- **Regulated goods** — quarantine-on-receipt defaults by class, release authority roles, controlled-substance classification axis, recall reason codes and regulatory clock definitions, deviation and corrective-action states, electronic signature requirements by action
- **Multi-entity** — intercompany accounting codes and matched-posting rules, transfer pricing method placeholders, elimination dimension values, group currency and translation rate types
- **Channel sales** — order intake states for automated upstream systems, hold types with release authority, cancellation cut-off definitions, prepaid settlement and clearing rules, notification events back to the originating system
- **Outsourced logistics** — third-party custody location behaviour, provider reconciliation cadence, custody-versus-ownership defaults, discrepancy handling

12.11 Applying a pack

1. Select a preset, then adjust capabilities individually
2. The system generates a change set containing everything the selection implies, and only that
3. Conflicts surface before anything lands: account ranges colliding with a legislation pack, duplicate codes, unmet capability dependencies
4. Required decisions are listed and **promotion refuses while any remain**: approval thresholds, sequence prefixes, costing method per class, tolerance values, mandatory dimension choices
5. Preview the diff, promote
6. The tenant records pack identifiers and versions

Re-application is additive: a tenant that skipped manufacturing at onboarding can add it later, and the change set contains only what is missing.

12.12 Maintenance and neutrality review

- Pack updates ship as new versions; existing tenants are unaffected. A tenant may review a diff against a newer version and adopt selected changes as a change set
 - **Neutrality review is a release gate.** Before any pack version ships it is checked for content traceable to a specific organisation — no customer, site, product, account, department or threshold taken from a live tenant. If a tenant's configuration suggests a good default, the pattern is generalised and the artefact rebuilt from neutral inputs
 - Every value carries a provenance note naming the standard or practice it derives from, so the review is checkable rather than trusted
-

12.13 Acceptance

A new organisation, having chosen the Standard preset and bound one legislation pack, can — without further configuration and without engineering involvement:

- Raise a requisition, route it through an approval band, convert it to a purchase order, receive goods within tolerance, and match a supplier invoice
- Have every posting determined by rule with correct dimensions, and no suspense fallback
- Receive batch-controlled stock into quarantine, release it under named authority, and store it under container identity
- Count without freezing operations, and post a variance within tolerance
- Take a sales order, allocate globally then in detail, replenish a marshalling area, pick, despatch and invoice
- Answer a recall question for any batch
- Close a period with suspense empty and every posting traced to its rule version

Anything requiring a value the pack did not provide and onboarding did not ask for is a gap in the pack, logged against the product.

PART 14 — DEVICE OPERATIONS

The warehouse does not use the application described in Part 7. It uses a different application against the same functions: one task at a time, driven by scanning, operated with gloves on, in poor light, sometimes without a network.

14.1 Device classes

- **Ruggedised handheld terminal** — the primary device. Android, physical scan trigger, 4 to 6 inch portrait screen, worn or holstered. Assume Zebra TC-series or Honeywell CT-series ergonomics as the baseline
- **Ring or wearable scanner with wrist display** — hands-free picking; the display shows one instruction, the scanner confirms it
- **Vehicle-mounted terminal** — forklift work: putaway, bulk moves, replenishment. Larger screen, operated stationary, never while moving
- **Tablet** — supervision, quality inspection, receiving desks, goods-in booking
- **Fixed station** — packing benches and despatch desks, with a keyboard, a scanner and a label printer
- **Voice** — picking by spoken instruction and confirmation, where the operation justifies it

One codebase serves all of them. Layout adapts; the task flow does not change between devices, because an operator moving from handheld to vehicle terminal should not relearn the work.

14.2 Design constraints

These are not stylistic preferences. Each comes from a physical condition of the environment.

- **One decision per screen.** Never two questions at once. The operator is standing, holding something, and being interrupted
- **Scan first, type never.** Every identifier is scanned. Keyboard entry exists as an exception path, always requiring a reason where it substitutes for a scan of something that should have been scannable
- **Minimum 48 pixel touch targets, 64 preferred.** Gloves are assumed, not accommodated
- **High contrast, large type, no colour-only meaning.** Warehouse lighting varies, screens are viewed at arm's length in cold stores through condensation, and colour blindness is common
- **Portrait, thumb-reachable.** Primary actions sit in the lower third; destructive actions never sit where a thumb rests
- **Confirmation by scan, not by button.** Confirming a pick by scanning the location and the product is verification; confirming by tapping "yes" is not
- **No modal dialogs, no nested navigation.** The back path is always one step and always the same
- **Audible and haptic feedback on every scan** — distinct signals for accepted, rejected and complete. In noise, haptics carry
- **Session survives the device sleeping, dropping signal, or being holstered mid-task**

14.3 Task set

Each task is a self-contained flow with a defined start, a defined completion, and a defined abandon path. The list is the standard warehouse task inventory.

Inbound - Goods-in booking: confirm arrival against expected receipt - Receipt against purchase order or advance shipping notice: scan product, capture quantity, batch, expiry, serial where controlled - Receiving discrepancy: short, over, damaged, wrong product, temperature excursion — each with reason capture and photograph - Handling unit build: create a unit, add contents, nest under a parent, apply identity - Putaway: system-directed to a suggested location, or confirmed where the operator chooses; scan location to confirm

Stock - Stock enquiry: scan a product, location or handling unit and see position, batches, expiry, allocation state - Internal move: from location to location, or handling unit to location - Replenishment: directed tasks from bulk to pick face or marshalling area - Cycle count and stocktake: blind or informed, count by unit, by handling unit, or hybrid; recount on tolerance breach - Adjustment and write-off: reason mandatory, approval routed where above threshold - Batch actions: quarantine, release where authorised, block, split, merge

Outbound - Pick: order, batch, cluster, zone or wave, directed by location sequence, confirmed by scanning location then product then quantity - Short pick: reason capture, automatic exception raised, replenishment triggered where stock exists elsewhere - Pack: scan-to-verify contents against the order, carton selection, weight capture, packing label - Marshalling: move completed units to the marshalling area, confirm by scan - Despatch and loading: build the load, scan units onto the vehicle, capture carrier and consignment, no despatch without a scan - Returns receipt: scan the return, capture condition, route to disposition

Production - Component issue against a works order, batch-controlled and scan-confirmed - Operation booking: quantity completed, quantity scrapped with reason, time capture - Finished goods receipt with batch creation and label print

Quality - Inspection execution: sampling, results capture, photograph - Disposition: accept, accept under concession, reject, return, destroy

14.4 Scanning standards

Standards, not conventions, because scanned data arrives from suppliers and carriers who follow them.

- **GS1-128** with application identifiers parsed on receipt: 00 serial shipping container code, 01 global trade item number, 10 batch or lot, 17 expiry, 21 serial, 37 count. A single scan populates several fields, and the parser is shared, not per-screen
- **GS1 DataMatrix** for two-dimensional marking, including pharmaceutical unit packs
- **SSCC** as the identity of a handling unit, generated by the platform for units it creates
- **EAN-13, UPC-A, Code 128, Code 39** accepted for legacy and internal marking
- **Symbology and identifier rules are configuration:** which symbologies are accepted at which step, which application identifiers are mandatory for which product class, and what happens when a required identifier is absent
- Unrecognised barcodes are rejected with the scanned value shown, never silently ignored
- Where a supplier's barcode does not carry a needed field, the exception path captures it with a reason, and the supplier's data quality is recorded against them

14.5 Offline behaviour

A network drop in a cold store must not stop the work.

- **What works offline:** task lists already assigned, scanning and validation against cached master data, capture of counts, picks, moves and putaways into a local queue
- **What does not:** anything requiring a decision only the server can make — allocation, approval, release of a batch from quarantine, or any action against data the device has not cached
- **Store and forward** with idempotency keys on every queued action, so reconnection never duplicates
- **Conflict handling on reconnection** is explicit: an action that is no longer valid (the stock moved, the order cancelled) surfaces to the operator as an exception with the reason, and never posts silently or silently disappears
- The device shows its own state plainly: connected, queued with a count, and syncing. An operator must always be able to see whether their work has landed

14.6 Identity on shared devices

- Devices are registered and bound to a site; an unregistered device cannot transact
- Fast user switching: badge scan or PIN, because a shared terminal changes hands many times per shift
- Sessions are short and re-authenticate on resume; every action attributes to the user who performed it, never to the device
- Supervisor override is a separate authenticated act recorded against both users, used for approvals above an operator's threshold

14.7 Performance budgets

Stated because they are the difference between adoption and abandonment:

- Scan to on-screen response: under 300 milliseconds for cached validation
- Task list load: under 2 seconds
- Screens per pick line: one
- Taps per confirmed action beyond scanning: zero in the normal path

PART 15 — OUTPUT AND COMMUNICATIONS

One subsystem produces everything the platform emits: printed documents, labels, emails, notifications and machine messages. Treating them separately is how organisations end up with four template systems and no archive.

15.1 The common model

- **output_template** — versioned, effective-dated, bound to an output type and a rendering engine, with data binding declared against the canonical model rather than against a query
- **output_request** — an intent to produce something: template, data reference, destination, locale, copies, triggering event, requesting user
- **output_render** — the produced artefact: bytes, format, checksum, template version, data snapshot reference, timestamp
- **output_delivery** — the attempt to get it somewhere: destination, status, retries, confirmation or failure reason
- Every rendered output is archived and retrievable by its document reference, retained per the legislation pack's retention rule, and its template version recorded so a document can always be reproduced exactly as issued

15.2 Document rendering

- Documents render from templates to PDF for anything a person or an authority reads
- Locale is resolved per Part 3: the recipient's language for customer-facing documents, the company's reporting language for statutory output
- Templates are branded per organisation through the same resource and branding layer, so branding is not a fork of the template
- Standard document set: purchase order, order acknowledgement, goods received note, picking list, packing note, delivery note, commercial invoice, credit note, pro-forma invoice, statement, remittance advice, transfer note, works order pack, certificate of analysis, customs documentation, recall notice
- Documents that carry a signature render its manifestation on the face of the document: who signed, in what capacity, when, and what the signature meant
- Reissue is distinguishable from original: a reprinted document is marked as a copy, and the reissue is recorded

15.3 Label generation

Labels are not small documents. They render to printer command languages, at fixed physical sizes, and correctness is measured by whether a scanner reads them.

- **ZPL** as the primary label language, with EPL and IPL supported for legacy estates
- Label stock and resolution are properties of the printer, not the template: the same template renders at 203 and 300 dots per inch without a second template
- Standard label set: pallet or handling unit label carrying SSCC, carton label carrying GTIN and batch, product label, batch label, location and bin label, returns label, quarantine label, despatch and carrier label
- **GS1 layout compliance** for handling unit labels: correct application identifier structure, human-readable interpretation beneath the barcode, and quiet zones respected
- **Barcode verification** is part of the definition of done: a template ships with a test render and a decode check, so a label that will not scan cannot be promoted
- Carrier labels are produced from carrier integrations where the carrier mandates their own format, and the platform stores the returned artefact rather than reproducing it

15.4 Print orchestration

- **printer** — registered per site, with type, language, resolution, physical location, default stock and queue address
- Routing rules bind an event and an output type to a printer or printer group, by site, workstation, packing line or user
- Printing is a queued, retried operation with a monitored backlog, not a request that succeeds or vanishes
- **Queue health is a first-class signal**, alerting before the operation notices: printer offline, queue depth above threshold, no successful print in a window
- Reprint is self-serve within permission, always audited, and always marked as a copy
- Print at the point of use: an operator's device requests, the platform routes to the printer nearest that task rather than to a default

15.5 Email

The one channel with reputational and deliverability mechanics of its own.

- **Sender identity** — per organisation sending domain with SPF, DKIM and DMARC alignment. A tenant sending as its own domain is verified before it can send; until then it sends from a platform domain with a reply-to
- Transactional and operational categories are separated at the sender level so a bounced marketing-style message cannot damage delivery of an invoice
- **Bounce, complaint and suppression handling** — hard bounces suppress automatically, complaints suppress permanently, and the suppression list is visible and auditable. Sending to a suppressed address is refused
- Attachments are produced by the output subsystem, not composed separately, so an emailed invoice is the same artefact as the printed one, with the same archive entry
- Every message is archived with its rendered body, recipients, and delivery outcome
- **Standard email set**: order acknowledgement, purchase order to supplier, remittance advice, invoice, statement, dunning sequence, despatch confirmation with tracking, return authorisation, approval request and reminder, recall notification, report subscription
- Inbound email is not a supported intake path by default; where an organisation needs it, it arrives as a registered integration adapter with parsing rules, never as an unstructured mailbox

15.6 Notifications

- Channel abstraction over email, in-app, mobile push, Slack, Teams, SMS and webhook, with routing as configuration
- Routing binds an event and a severity to an audience — a role, a department, a named user, or the owner of the affected object — never to a hard-coded address list
- **Digest and escalation** — related alerts batch into a digest at a configured cadence; unacknowledged alerts above a severity escalate on a timer
- **Quiet hours per user and per channel**, with an override severity that always breaks through
- Per-user preferences within the bounds the organisation permits, so an operator can choose push over email but cannot switch off an alert their role requires
- Delivery is tracked: sent, delivered, read where the channel supports it, and failed with reason
- In-app notification is the fallback that always works and never gets suppressed, so no alert is lost because a channel was misconfigured

15.7 Configuration and governance

- Templates, printers, routing rules, sender domains and notification bindings are all configuration: versioned, effective-dated, previewed and promoted through change sets like everything else
- **Preview before promotion is mandatory for any output template**: a rendered sample against representative data, and for labels a decode check
- Test sends and test prints are available in the test environment against the same templates, so nothing is first proved in live
- Output volume, failure rates and queue depth are reported alongside job health in the assurance surface

15.8 What this adds to the starter packs

Part 12 lists notification templates and document and label templates as starter content. That list stands; this part is the subsystem beneath it. Two additions to the packs follow from it:

- A **printer profile** starter set: common label sizes and resolutions, so a new site registers printers by choosing a profile rather than describing one
- A **sender identity checklist** in onboarding: the DNS records an organisation must publish before it can send as its own domain, presented as a task with verification rather than as documentation

PART 16 — RELEASE AND ENVIRONMENTS

Everything so far describes a product. This part describes it as a running service, because the specification currently stops at the point where software has to reach a machine.

16.1 The environment ladder

- **Development** — per engineer, disposable, built from an empty database by applying the host bootstrap and every migration in order. If it cannot be rebuilt from nothing, it is not an environment
- **Continuous integration** — created and destroyed per build, from empty, running every structural assertion and adversarial suite. It is the only place where a green result means anything, because it is the only place with no accumulated state
- **Test** — long-lived, one per deployment, carrying every organisation's test environment and the starter packs. Where configuration is authored and change sets are validated
- **Live** — the operating service

An organisation's own test environment is not a separate deployment. It is a tenant-scoped environment inside the same instance, which is what makes promotion a configuration operation rather than a data migration.

16.2 Schema release

- **Migration-first, always forward.** Every schema change is a numbered, timestamped migration applied in order, each in a single transaction. There are no console edits to structure, in any environment
- Down migrations are not written. Reversal is a new forward migration, because a down migration is tested exactly once and then relied upon at the worst possible moment
- A migration that adds a callable function must register it and pass the boundary assertion in the same migration, so an ungoverned entry point cannot survive its own transaction
- **The host contract is explicit.** A single bootstrap script states everything the platform expects of its host — extensions, roles, the identity function — so the database is portable rather than bound to one provider by accident

16.3 Application release

- Trunk-based with short-lived branches; the build is the artefact, promoted between environments rather than rebuilt per environment
- The pipeline gates on: type check, lint, unit and component tests, a build, the full database suite from empty, and an authenticated route sweep. **Every one of these covers the application as well as the database** — a pipeline that verifies only the schema leaves the half users touch unverified
- Deployments are versioned and immutable, with the previous version retained and revertible without a rebuild

- Schema and application versions are compatible across at least one release in each direction, so a deployment and its rollback never require a simultaneous schema change

16.4 Configuration release

Distinct from software release, and already specified in §4.11: change sets, previewed, approved, promoted, revertible. The two must not be confused. Software changes what the platform can do; configuration changes what an organisation has asked it to do. A release that requires both is sequenced explicitly, software first.

16.5 Backup, restore and continuity

- Continuous backup with point-in-time recovery to the granularity stated in §9.2
- **Restore is proved by drill, not by log.** A scheduled restore into an isolated environment, verified by running the invariant assertions against the restored data, on a stated cadence. A backup that has never been restored is a hope
- Per-tenant restore is possible without affecting other tenants, because a shared-instance restore that requires downtime for everyone is not a usable remedy for one organisation's mistake
- Continuity plan states what happens if the host provider is unavailable, including the portability position the host contract makes possible

PART 17 — SUPPORT AND INCIDENT MANAGEMENT

The platform already carries an audited cross-tenant support mechanism. This part supplies the operating model around it, because a capability without a discipline is a liability.

17.1 Support access discipline

- Support access is invoked against a named reason and, where one exists, a reference to the organisation's own request. Curiosity is not a reason
- Access is time-bounded and expires automatically; extension is a fresh act, recorded
- The organisation can see, at any time, every occasion its data was entered by platform staff, by whom, when, for how long and why. This is a screen, not a request
- Read-only by default. Any support action that writes is performed as an explicit, separately recorded act, and the organisation's own audit shows platform staff as the actor rather than one of its users
- Standing access does not exist. Every session is a new invocation

17.2 Severity and response

Severity is defined by operational effect, not by how loudly it is reported:

- **Severity 1** — the organisation cannot transact: goods cannot be received, picked, despatched or invoiced; or data integrity is in question
- **Severity 2** — a significant function is unavailable or producing wrong results, with a workaround
- **Severity 3** — a defect with a workaround and no operational stoppage
- **Severity 4** — a question, a request, or a cosmetic defect

Each carries a stated response and update cadence, published rather than negotiated case by case. Severity is agreed with the organisation, and disagreement escalates rather than being resolved unilaterally.

17.3 Incident management

- An incident is declared, not drifted into. Declaration names a commander, a communications owner and a scribe, however small the team
- **Communication is on a timer**, not on progress: an update at a stated interval even when the update is that nothing has changed, because silence is what erodes trust during an outage
- A shared platform means an incident affecting one tenant may or may not affect others. The first question after containment is scope, and communication is scoped accordingly — never a broadcast that alarms unaffected organisations, never silence toward affected ones
- **Data integrity incidents have their own path.** Where postings, stock positions or documents may be wrong, the immediate action is to stop the affected path rather than to keep trading with suspect numbers, and the remedy is a reversing correction with an evidence pack, never a quiet fix
- A blameless post-incident review is written for every severity 1 and 2, shared with affected organisations, and its actions tracked to completion

17.4 Status and disclosure

- A status page states current and historical availability, maintained by the platform, not by inference
 - Planned maintenance is announced against the windows in §9.2
 - Security incidents follow a disclosure path with stated timelines, including regulatory notification obligations that fall on the organisation as controller and the platform as processor
-

PART 18 — COMMERCIAL AND ENTITLEMENT

The service commitments in Part 9 and the lifecycle in Part 2 imply a commercial relationship the specification never describes. Product behaviour must exist behind those terms or they cannot be honoured.

18.1 Subscription and entitlement

- **subscription** — an organisation's plan: term, renewal, currency, and the entitlements it grants
- **entitlement** — what a plan permits, expressed against the same objects the product already has: which capabilities (§12.2) are available, how many users, how many companies and sites, transaction volume bands, retention period, environment count, support severity commitments
- Entitlement is enforced in the database alongside permission, not in the interface. An organisation cannot exceed its plan by calling a function directly
- Exceeding an entitlement produces a named refusal and a notification to the organisation's administrator, never a silent degradation and never a surprise invoice

18.2 Metering

- Usage is measured from events the platform already produces: documents posted, movements recorded, active users in a period, storage held, messages sent
- Meters are visible to the organisation continuously, in the same units the plan is expressed in, so the number on an invoice is one the customer has already seen
- Metering data is tenant-scoped and retained independently of operational data, so a purge does not destroy the billing record — and equally, so the billing record holds no operational detail beyond counts

18.3 Lifecycle and non-payment

The lifecycle in §2.5 gains its commercial triggers:

- **Grace** — service continues, administrators notified, no functional change
- **Restricted** — writes refused, reads and export retained. An organisation that cannot pay must still be able to retrieve its records; withholding data is not a collection method
- **Suspended** — access withdrawn, data intact, restoration immediate on resolution
- **Deletion** — only on the organisation's instruction or after a stated period following termination, always with export offered first, and always with the key destruction in §2.5

18.4 The service commitment

- Availability, recovery objectives, support response and data protection terms are stated in one place, derived from Part 9 and Part 17 rather than written separately, so the contract and the product cannot drift apart
- Where the platform fails its commitment, the remedy is stated in advance
- Sub-processors are listed, with notification before any addition, because the organisation carries controller obligations that depend on knowing them

PART 19 — REPORTING AND ANALYTICS SERVICES

§7.10.2 lists a report catalogue and dashboards. This part is the subsystem beneath it — the same gap printing had before Part 15.

19.1 The governed query layer

- Reports read from **governed views**, never from tables and never from ad-hoc SQL embedded in a report definition. A view is a versioned, named contract with a stable shape
- Every governed view carries tenant scoping and honours the caller's company, site and, where configured, department scope. **Report output is scoped by the same rules as operational data** — a report is not a way to see what a screen would refuse
- Row-level scoping is applied in the view, not in the report, so a new report inherits it automatically and cannot forget it
- Views are the single definition of a metric (§7.10.2's KPI framework), so a figure means the same thing in a dashboard, a scheduled pack and an external tool

19.2 Report definitions

- **report** — versioned, effective-dated: governed view, parameters, columns, grouping, default sort, permitted output formats, required permission
- Parameters are typed and validated; a report cannot be parameterised into reading outside its scope
- Reports are configuration and promote through change sets, so a tenant's own reports move between its environments the same way everything else does
- Every rendered report records its definition version, parameters, the user, and the time — so a figure quoted in a meeting can be reproduced exactly

19.3 Execution and scale

- Interactive execution has a time budget and a row cap; beyond either, the request becomes a scheduled extract rather than failing
- Heavy reporting reads from a replica or a read-optimised projection, never from the transactional path in a way that can slow a despatch scan

- Long-running extracts are queued, produce an artefact through the output subsystem in Part 15, and notify on completion

19.4 Scheduled distribution and packs

- Subscriptions by user or role, with delivery through Part 15 so a scheduled report is archived and traceable like any other output
- Assembled packs — period close pack, weekly operating pack, board pack — are a defined artefact with a manifest of what they contain and the as-at time of each figure

19.5 External analytics

- A read-only analytics contract: governed views exposed for external business intelligence tools, with credentials that are tenant-scoped, permission-scoped and separately revocable
- **The contract is versioned and deprecated on notice**, because an external tool built against a view becomes a dependency the platform must not break silently
- Bulk export for warehousing is a scheduled, incremental extract of governed views, never direct database access
- Natural-language querying operates over the same governed views and is read-only, so it inherits every scope rule rather than needing its own

PART 20 — DATA MIGRATION AND IMPORT

\$9.6 promises coexistence and progressive cutover; \$7.1 mentions controlled import pipelines. Neither says how data actually arrives, which is the single most common way an ERP implementation fails.

20.1 The import pipeline

Every load, whether a one-off migration or a routine bulk update, follows one path:

- **Define** — a mapping from a source shape to the canonical model: source field, target field, transformation, default, and whether it is required. Mappings are versioned configuration, not scripts
- **Stage** — the file lands in a staging area, parsed and typed, with nothing yet touching operational data
- **Validate** — every row checked against structural rules, referential integrity, and business rules. Output is a per-row report naming the row, the field and the reason, not a single failure message
- **Preview** — what would be created, updated, skipped and rejected, in counts and in sample, before anything is committed
- **Load** — committed as a single named batch, with every created object tagged with the batch identifier
- **Reconcile** — a report comparing source totals to loaded totals: record counts, quantity sums, value sums. A migration is not complete because it ran; it is complete because it reconciles
- **Reverse** — a batch can be reversed while its objects remain untouched by subsequent activity, and the reversal is itself recorded

20.2 Migration sequencing

The order is not a preference; each stage depends on the last:

- Structure — companies, sites, locations, departments, calendars
- Configuration — accounting codes, determination rules, approval bands, numbering, policies
- Reference master data — units, classifications, business partners, products
- Relationships — supplier catalogues, bills of material, routings, price lists
- Opening balances — stock on hand by product, batch, location, owner and condition, with as-at dating; open documents; ledger opening balances
- Historical data — only what is genuinely needed, because history is where migrations expand without limit

20.3 Opening balances

- Stock arrives as an opening movement with an explicit as-at date and a reason code, so the ledger's first entry is honest rather than implied
- Batch attributes, ownership and custody arrive with the stock, not afterwards, because retrofitting them means adjusting every position
- Ledger opening balances load per company, per ledger, with dimensions, and must balance before the load commits
- Open documents arrive in their true state, with the portion already fulfilled recorded, not as fresh documents

20.4 Parallel running and cutover

- **Parallel run is a comparison, not a duplicate entry exercise.** The incumbent system's positions are ingested on a cadence and compared to the platform's, per domain
- Divergences are classified — timing, mapping, genuine defect, incumbent error — aged, and tracked to resolution. The classification matters: a timing difference that closes by itself is not the same signal as a mapping error
- Cutover is evidence-gated per domain: a stated period of clean reconciliation, not a date in a plan
- Each domain carries its own cutover state and its own rollback, so one domain reverting does not unwind the others

- After cutover the incumbent's identifiers remain as external references, so history stays navigable

20.5 Data quality

- Quality is assessed before migration, not discovered during it: completeness by field, validity against vocabularies, duplication, orphaned references
- The report goes to the organisation as a work item with owners, because the platform cannot invent data the source lacks
- Loading known-bad data is permitted where it is unavoidable, but explicitly and with the defects recorded, never silently normalised

PART 21 — ACCESSIBILITY AND INCLUSIVE DESIGN

Part 14 addresses gloves, glare and noise. This part addresses the desk application, where the constraints are different and the obligations are legal.

21.1 The standard

- **WCAG 2.2 level AA** for the desk application, as the level referenced by UK public sector procurement and increasingly by enterprise buyers
- Conformance is claimed in an accessibility statement that is accurate, including known exceptions, rather than aspirational
- Accessibility defects are defects. They are triaged on the same severity scale as any other, not in a separate backlog that never clears

21.2 What this requires in practice

- Every action reachable and completable by keyboard alone, in a logical order, with visible focus
- Semantic structure and accessible names on every control, so screen readers announce purpose rather than position
- Contrast minimums met, and **no meaning carried by colour alone** — a status is a word and a shape as well as a colour
- Text resizes to 200 per cent without loss of function; layout reflows rather than requiring horizontal scrolling
- Errors identified in text, associated with the field, and describing the correction — which the friendly-error layer already does for refusals and must also do for validation
- Motion respects reduced-motion preferences; nothing conveys state by animation alone
- Time limits are avoidable or extendable, which matters where an approval or a count session times out

21.3 Beyond conformance

- The device application in Part 14 inherits what applies — contrast, target size, text scaling — and is tested with gloves and in cold-store conditions rather than at a desk
- Terminology plainness is an accessibility property, not only a style one. The vocabulary in Part 3 exists partly for this reason
- Testing includes assistive technology, not only automated checks. Automated tooling finds a minority of real barriers

PART 22 — GUIDANCE AND ADOPTION

This platform refuses things by design — unmatched postings, unapproved changes, allocations outside scope. A refusal a user cannot act on is indistinguishable from a broken system, so guidance is a product surface rather than documentation.

22.1 Refusals that teach

- Every refusal carries three things: what was refused, why in plain language, and the next action — the change set to promote, the rule to configure, the permission to request, the person to ask
- Technical detail is retained and available, collapsed, for whoever needs it
- Where a refusal is common enough to be predictable, the interface prevents the situation rather than explaining it afterwards
- Refusal messages resolve through the resource layer like everything else, so an organisation can add its own local guidance to a standard message

22.2 In-product guidance

- Contextual help against a screen, an action and a configuration object, sourced from the same content set as the published documentation so the two cannot diverge
- **First-run guidance for each role**, not a single product tour: what a warehouse operative needs on day one is not what a finance manager needs
- Configuration screens explain the consequence of a setting, not only its name — a threshold field states what happens above and below it
- Empty states carry the action that fills them, because a new organisation's first experience is a series of empty screens

22.3 Sandbox and practice

- Every organisation has a sandbox environment seeded from starter packs, where a person can complete a real task without consequence

- Training scenarios are configuration: a named starting state, a task, and a completion condition, so an organisation can build its own
- The demo seed is refused in a live environment by the platform, not by a warning in a guide

22.4 Adoption signals

- The platform reports, to the organisation about itself, where people are struggling: refusals by type and frequency, tasks abandoned mid-flow, approvals ageing, features enabled but unused
- This is the organisation's own operational data, used to improve its configuration and training — not behavioural surveillance of individuals, and not exported across tenants

PART 23 — DECISIONS REGISTER

Choices already made, with the reasoning that produced them. A decision recorded here is settled: it may be revisited deliberately, but not reopened by accident or drifted away from in a later build. Each states what was decided, why, and what it costs.

D1 — Tenancy is the outermost scope, not a feature

Every business object, configuration object, event, document, audit record, file and job carries a tenant. There is no unscoped operational table. **Why:** isolation added later cannot be proven; isolation designed in can be asserted adversarially on every build. **Cost:** every query path and every job carries context plumbing.

D2 — State is derived, never stored as an editable balance

On-hand, allocated, available, document position and subledger balances are computed from an append-only ledger. Corrections are reversing entries. **Why:** it makes reconciliation, audit, recall and replay structural rather than bolted on, and it is the structural answer to phantom stock created by upstream deletion. **Cost:** projections and their reconciliation checks are mandatory rather than optional (\$6.6).

D3 — One user, one organisation

Cross-tenant identity does not exist. A person needing access to two organisations holds two accounts. **Why:** identity spanning tenants is the shortest path to a leak that row security cannot catch. **Exception, deliberate and narrow:** platform staff may enter an organisation under an audited, time-bounded support access recorded in the platform log. That is support access, not identity.

D4 — Configuration is data, and live configuration cannot be edited in place

Changes accumulate in change sets, are previewed, approved, promoted, and are revertible to a versioned snapshot. **Why:** a change nobody approved and nobody can explain is the failure this platform exists to prevent. **Cost:** an administrator cannot make a quick fix in live, by design.

D5 — Legislation is data, not an installed pack

Jurisdictions arrive as versioned, effective-dated configuration bundles bound per company, evaluated by rule. No branch in code tests a country. **Why:** activating a market should not be an outage, a vendor dependency or a consultancy engagement. **Cost:** a conformance suite per jurisdiction is required, not optional.

D6 — No user-facing literal anywhere

Every string resolves through a resource key with locale fallback. **Why:** retrofitting a resource layer touches every screen, and per-country duplication of configuration is the largest hidden cost in multi-country setup. **Cost:** discipline from the first screen onward.

D7 — Accounting identity is a code on the product, not an account

A product carries an accounting code; accounts are resolved by a determination matrix taking transaction type, accounting code, business partner class, site, company, ledger, legislation and reason code. **Why:** an account held directly on a product must be re-coded per company, per ledger, per transaction type. **Superseded:** the earlier notion of assigning GL codes directly to products. Not to be reintroduced.

D8 — No default-to-suspense

An unmatched posting is refused and named, and a coverage assertion enumerates gaps before promotion. **Why:** silent fallback to suspense is how a class of goods posts to the wrong account for a quarter without anyone noticing.

D9 — Ownership is separate from custody

Every position and movement carries an owner and a custody party. Valuation follows ownership; operational visibility and counting follow custody. **Why:** third-party logistics, consignment in both directions, and contract manufacturing are ordinary in the target profile, and retrofitting this touches every movement row.

D10 — Batch attributes are amendable without moving stock

Expiry, use-by, retest, supplier lot and status are amended by evented amendment with approval proportionate to the field. No stock movement is generated. **Why:** issuing stock out and receiving it back fabricates history that never happened, disturbs valuation, and breaks the genealogy a recall depends on.

D11 — Counting does not freeze operations

A count records a counted quantity at a timestamp; expected quantity is reconstructed as at that moment and subsequent movements replayed on top. Locking, where genuinely needed, is the narrowest scope and soft by default. **Why:** counts that stop fulfilment get deferred, and accuracy suffers for the sake of not interrupting despatch.

D12 — Handling unit identity depth is configuration

Containers nest without limit; the level at which identity is captured is policy per product class, site and process step, with count method bound to the same policy object. **Why:** changing from per-box identity to master pallet identity is a configuration change, not a development project. Historical units keep the policy under which they were created.

D13 — Allocation is two-stage, and scope is policy

Global allocation reserves quantity at site level; detailed allocation commits specific stock under a configurable location scope. Where global succeeds and detailed fails, the cause is classified and, where stock exists outside scope, an internal replenishment task is raised. **Why:** an order that is globally allocated but not detail-allocated is not a shortage, and treating it as one leaves a backlog nobody owns.

D14 — Printing is downstream of successful allocation

Documents and labels are produced only for orders with committed stock in the marshalling area. **Why:** it structurally removes the failure mode where labels exist for an order that cannot be picked.

D15 — Product identity is opaque; the readable code is derived

The internal key is a meaningless surrogate. Classification is structured data from controlled vocabularies. The composed code is a rendering of that classification, unique and stable, flagged when it diverges from the attributes. **Why:** codes that encode meaning eventually lie, and correcting them breaks the audit trail.

D16 — Approval routing is department and threshold driven, with named assignment alongside

Users belong to departments; bands route by value; named assignments override or prepend where a specific approver is required. Resolution order is named assignment, department band, company fallback, and every resolution records the rule version that produced it. **Why:** it preserves the routing organisations already operate rather than imposing a new model.

D17 — The department object is single

The department used for approval routing is the same object used as the analysis code for reporting. **Why:** two lists diverge, and then operations and reporting disagree about who spent what.

D18 — Artificial intelligence proposes; it never applies, and never decides a transaction

Configuration intelligence authors, explains, analyses impact and generates tests, always as a diff requiring named human approval, validated in test first. Postings, allocations, tax determination and stock movements are executed by deterministic rules only. **Why:** in a regulated estate, the line between authoring a rule and deciding a transaction is what keeps the system auditable.

D19 — All outbound writes pass through one gateway

Commands carry idempotency keys, approval state, dry-run capability and per-scope flags; no module holds credentials or calls an external system directly. An outcome that is unknown enters an explicit ambiguous state rather than being retried blindly. **Why:** a bad write to a live external system is the highest-consequence failure available to this product.

D20 — Capabilities are switchable, and cannot be switched off dishonestly

Behaviour arrives as named capabilities, effective-dated and switched through change sets, gating navigation, fields and rules together. A capability with live data cannot be disabled — the attempt is refused with a named reason. **Why:** comprehensiveness is only tolerable if unused capability is genuinely absent rather than merely hidden.

D21 — Starter content is neutral by construction, and neutrality is a release gate

Packs derive from published standards, statutory structures or generic practice. No artefact derived from a live organisation enters product content; where a pattern is worth generalising, it is rebuilt from neutral inputs. Every value carries a provenance note. **Why:** it is what makes the product genuinely reusable, and what keeps any one organisation's configuration out of every other organisation's system.

D22 — Model names are stable; product language is data

The schema keeps `tenant`, `party`, `item`, `principal`. The product surface says organisation, business partner, product, user. **Why:** renaming tables costs migrations and buys nothing, while renaming on screen costs a glossary entry.

D23 — The foundation is finished when a second organisation can be onboarded by someone who has never seen the

first

This is the acceptance test for the whole of this document, not a slogan. **Why:** every other measure of completeness can be satisfied by a system shaped around one customer.

D24 — Confirmation in the warehouse is by scan, not by tap

Every operational confirmation on a device is made by scanning the thing being confirmed — the location, the product, the handling unit. A tap is an acknowledgement, not a verification. **Why:** the value of a directed task is the check it performs; a button press confirms only that someone was holding the device.

D25 — One output subsystem produces everything the platform emits

Documents, labels, emails, notifications and machine messages share one template model, one render record, one delivery record and one archive. **Why:** separate paths produce separate template systems, and then a document exists in three versions with no authoritative copy. **Consequence:** an emailed invoice is the same artefact as the printed one, with the same archive entry.

D26 — An output template is not promotable until it has been proved

Every template ships with a rendered sample against representative data, and every label additionally with a decode check. **Why:** a label that will not scan is not a cosmetic defect; it stops the operation, and it is discovered at the packing bench rather than in review.

D27 — Down migrations are not written

Schema reversal is a new forward migration. **Why:** a down migration is tested once and then relied upon at the worst possible moment. **Cost:** reversing a schema change takes as much care as making it.

D28 — A backup that has never been restored is a hope

Restore is proved by scheduled drill into an isolated environment, verified by running the invariant assertions against the restored data. **Why:** backup success logs record that a file was written, not that a business can be recovered.

D29 — An organisation that cannot pay can still read and export

The restricted state refuses writes and retains reads and export. **Why:** withholding a customer's own records is not a collection method, and the portability obligation does not lapse with an invoice.

D30 — Reports are scoped by the same rules as screens

Row-level scoping lives in the governed view, not in the report definition. **Why:** a report must never become the route to data an operational screen would refuse, and a rule applied per report is a rule that will eventually be forgotten.

D31 — A migration is complete when it reconciles, not when it runs

Every load produces a reconciliation of record counts, quantity sums and value sums against source, and every batch is reversible while untouched. **Why:** the most expensive migration defects are the ones that loaded successfully and wrongly.

D32 — Cutover is evidence-gated, per domain

A domain moves when its parallel-run reconciliation has been clean for a stated period, not when a planned date arrives, and each domain carries its own rollback. **Why:** a date is a wish; a clean reconciliation is a fact.

D33 — Accessibility defects are defects

WCAG 2.2 AA is the standard for the desk application, and failures are triaged on the ordinary severity scale. **Why:** a separate accessibility backlog is a backlog that never clears.

D34 — A refusal must always name the next action

Every refusal states what was refused, why in plain language, and what to do about it. **Why:** this platform refuses by design, and a refusal a user cannot act on is indistinguishable from a fault.